CASE STUDY

npm Supply-Chain Attacks, 2018 to 2026: The Record and the Defenses

A TeamStarWolf case study. Published 3 November 2026; every web source checked as of that date, the lab capture on 4 November 2026 UTC.

Executive summary

npm is where the JavaScript community shares its work. A few maintainers, many of them volunteers, publish small packages that millions of projects pull in by name, often several levels deep, on developer laptops and build servers that also hold cloud keys and publishing tokens. That arrangement is what makes the ecosystem fast, and it is why a single stolen login or a single poisoned release reaches so far. This study follows eight years of that record: the 2018 event-stream handover, the 2021 account takeovers, the maintainer sabotage of 2022, state-linked fake packages and fake recruiters, the 2025 phishing wave against maintainers of the most-downloaded packages, the Shai-Hulud worms of 2025 and 2026, and the compromises of axios, TanStack and keyv in 2026. It then sets out what the registry and the package managers changed in response, and what a defender can run today to check, clean and harden a Node estate.

Three things stand out. First, the attacker rarely needs a vulnerability: almost every incident here is a legitimate publish by someone who should not have been able to publish, through a hijacked account, a phished token, a poisoned CI workflow or malware on a maintainer's own machine. Second, the payloads mostly ran at install time through lifecycle scripts, which is the behavior npm v12, pnpm and Yarn Berry have now switched off by default for dependencies. Third, most bad versions were found and pulled within minutes to hours, which is why release-age gates work (**Inferred**). Provenance alone does not: the 2026 TanStack releases were published through the project's own trusted-publisher binding and the keyv releases carried valid provenance signed by GitHub Actions [43] [47].

Key facts at a glance

Field	Value
Period covered	9 September 2018 (event-stream 3.3.6) to 8 October 2026 (tensorlake malware advisory) [1] [54]
Largest single waves	Shai-Hulud: 500+ packages removed by GitHub (September 2025) [70]; Shai-Hulud 2.0: about 700 to nearly 800 packages and 25,000+ repositories by Wiz's count (November 2025) [26]; keyv wave: at least 444 packages and 1,381 versions by Aikido's count (August 2026) [47] (**Confirmed**, vendor counts)
Most-downloaded packages hit	chalk, debug and 16 others with more than 2 billion weekly downloads combined (Aikido, September 2025) [21]; axios at 100 million weekly downloads (Socket, March 2026) [36] (**Confirmed**, vendor figures)
How publish access was gained	Social engineering of a maintainer handover, account hijack, phishing of maintainers, poisoned CI workflows, stolen tokens reused by worms, malware on a maintainer's PC, a malicious VS Code extension (Part 3)
Where payloads ran	Mostly lifecycle scripts at install; also on `require()`, in browser bundles and CDN builds, and through editor extensions and AI coding tools (Part 4)
Time on the registry	coa about 72 minutes (2021) [7]; Nx about 4 hours (2025) [18]; axios 1.14.1 under 3 hours (2026) [35]; Mini Shai-Hulud detected within 6 to 8 minutes of each publish (2026) [38] (**Confirmed**)
State-linked activity	GitHub tied a 2023 campaign to Jade Sleet/TraderTraitor with high confidence [12]; Socket attributes more than 338 malicious npm packages to North Korea's Contagious Interview operation [24]; ATT&CK lists G0082 APT38 as injecting malicious dependencies into the Axios and Mastra packages [162] (**Confirmed**); the axios maintainer's post-mortem names no actor, though in a later comment on the same issue the maintainer likened the approach to Google's write-up of UNC1069 [35]
Registry response	2FA enforcement from 2022 [60] [65]; provenance GA 2023 [68]; trusted publishing GA 2025 [69]; classic tokens revoked 2025-12-09 [73]; npm v12 blocks dependency install scripts by default from 2026-07-08 [79]; publish-time malware scanning from 2026-07-28 [82] (**Confirmed**)
MITRE coverage	ATT&CK v20.0 models the worms as S9008 Shai-Hulud and S9043 Mini Shai-Hulud, the latter used by G1056 TeamPCP [160] [161]; no ATT&CK campaign covers npm [166] (**Confirmed**)

Key findings

- **Publishing rights are the target, not code flaws.** In 2018 an attacker took over event-stream through social engineering, by npm's own account, and added flatmap-stream to steal wallet keys (**Confirmed** [1]). GitHub's Chief Security Officer cited the 2021 ua-parser-js, coa and rc takeovers, noted that such takeovers at times involved accounts without 2FA, and committed to enforcing 2FA (**Confirmed** [58]). In 2025 and 2026 the access came from phishing that impersonated npm (eslint-config-prettier, chalk and debug), from CI workflow injection (Nx, PostHog, TanStack) and from a RAT on the axios lead maintainer's PC (**Confirmed** [17] [20] [18] [30] [43] [35]).
- **Install scripts did most of the damage.** coa used a preinstall script, Shai-Hulud 2.0, Mini Shai-Hulud and the August 2026 keyv wave used preinstall hooks, and Check Point notes the 2.0 payload runs even when

installation fails (**Confirmed** [7] [28] [38] [48]). npm v12 now blocks dependency lifecycle scripts unless the root project allows them (**Confirmed** [79]). The 2026 node-ipc stealer, which runs on `require()`, and the chalk/debug browser clipper sit outside that default (**Inferred** [44] [21] [78]).

- **The worms turned one stolen token into hundreds of packages.** Shai-Hulud republished up to 20 of each victim maintainer's packages, pushed a GitHub Actions workflow and exfiltrated to public repositories (**Confirmed** [22]). CHAINDROP, in August 2026, spread only when it found an npm token with write access and `bypass_2fa` (**Confirmed** [48]).
- **Provenance helps detection, but it is not a safety verdict.** Nx detected its bad versions because they lacked provenance, and eslint-config-prettier users noticed releases that had no matching commits and no provenance (**Confirmed** [18] [17]). The TanStack releases were published through the project's own trusted-publisher binding, and the keyv releases had valid provenance signed by GitHub Actions (**Confirmed** [43] [47]). npm's documentation says provenance does not guarantee a package has no malicious code (**Confirmed** [121]).
- **Speed of removal is the defender's best ally.** pnpm's release note says such attacks are usually discovered and removed within an hour (**Confirmed** [90]); the timings in this record mostly agree. Every malicious version in the exposure-windows table (Part 6) was found or pulled within a day, so a one-day or seven-day release-age gate would have skipped it; event-stream, which sat on the registry from 9 September to 26 November 2018, is the exception a gate would not have caught (**Inferred** from [1] [7] [18] [35] [38] [43]).
- **State actors use the registry as a delivery channel, not only as a target.** Contagious Interview personas published packages whose loaders rebuild BeaverTail in memory, which then typically fetches InvisibleFerret (**Confirmed** [24]). GitHub found Jade Sleet inviting targets to repositories with malicious npm dependencies (**Confirmed** [12]).
- **Known-vulnerability scanners do not see fresh malware.** A malicious version has no CVE on the day it is published. Malware advisories in the GitHub Advisory Database come from the npm security team and the OpenSSF Malicious Packages repository, and Dependabot raises them only when malware alerts are turned on (**Confirmed** [149]).

Top recommendations

1. Keep dependency install scripts off: upgrade to npm 12 (or set `ignore-scripts`), keep the `allowScripts` list short and pinned to versions, and use pnpm or Yarn Berry defaults that block unreviewed builds [79] [107] [127] [130].
2. Add a release-age gate: npm `min-release-age` (days, npm 11.10.0 and later), pnpm `minimumReleaseAge` (minutes; default 1440 since pnpm 11, unchanged in the current docs [126]), Yarn `npmMinimalAgeGate` [74] [93] [130].
3. Install only what the lockfile says in CI (`npm ci`, `yarn install --immutable`), block git and remote-URL dependencies (`allow-git=none`, `allow-remote=none`, pnpm `blockExoticSubdeps`), and route installs through a proxy that can quarantine [117] [79] [126] [142].
4. Publish without long-lived tokens: trusted publishing over OIDC, "Require two-factor authentication and disallow tokens", staged publishing with a human 2FA approval, and stage-only tokens for automation [122] [77] [86].

5. Harden release workflows: no untrusted code under `pull_request_target`, untrusted values through environment variables, actions pinned to full commit SHAs or a workflow lockfile, read-only `GITHUB_TOKEN` by default [145].
6. Turn on Dependabot malware alerts and the dependency review action, and keep an SBOM per build [76] [150] [120].
7. When a wave lands, check every lockfile and global install root against the published indicator lists, then remove, reinstall from a clean lockfile, clear caches and rotate every token the machine could reach [32] [52] [23] (Part 7).

Scope and method

This study covers malicious code published to the public npm registry from September 2018 to 3 November 2026: hijacked or sabotaged legitimate packages, worms that spread through maintainers' tokens, and campaigns that publish their own malicious packages. It also covers the registry, CLI and package-manager controls added in response, and the commands a defender uses to check and clean a Node estate. It does not cover ordinary vulnerabilities in npm packages, except where a scanner's limits matter. Every source except the lab capture [175] was opened on 3 November 2026. Quotations are under 15 words. Each claim carries one of four evidence grades.

Grade	Meaning
Confirmed	Taken from a primary source that was opened: the maintainer or affected project, the registry operator (GitHub runs npm), the researcher or vendor's own report, a government agency, or the GitHub Advisory Database
Reported	Taken from press coverage or a third party relaying someone else's finding, not independently corroborated
Demonstrated	A research proof of concept or reproduction exists, with no known malicious use
Inferred	The study's own reasoning from the cited facts, labelled as such

Numbers are reported as each source states them, and download counts and package counts are marked as the vendor's own figures. Times are given in the time zone the source uses. Where an advisory date may differ from the publish date of the bad versions, the study says "advisory date".

The commands in Part 7 come from each tool's own documentation and are marked Confirmed when the page documents the command or flag. A few lines are the study's own composition from documented flags; they are marked Inferred and have not been run for this study. A separate lab run recorded a subset of the check, audit and remove steps on a small Node tree inside a throwaway container; its screenshots and what they show are in Lab captures at the end of Part 7 [175].

Part 1: How npm trust works

The attacks in this record use ordinary registry behavior. None of it is a bug.

Names, tags and scopes

- Publishing sets the `latest` dist-tag unless `--tag` is given, and `npm install <pkg>` with no version installs `latest`. No other tag has special meaning to npm (**Confirmed** [99]). A hijacked account that publishes a new version therefore reaches everyone who installs without a pinned version or lockfile.
- Only the owning user or organization can publish under an `@scope`. Scoped packages are not public by default and need `--access public` on the first publish, and a scope can be bound to one registry (**Confirmed** [100]).
- Dependency confusion: when a resolver checks both an internal and a public index, it takes the higher version, so a public package with a high version number can stand in for an internal name. In February 2021 Alex Birsan uploaded packages under unclaimed internal names to npm, PyPI and RubyGems; they were detected inside more than 35 organizations, and Shopify and PayPal each paid a $30,000 bounty (**Demonstrated** [3]). His proof packages called home from a preinstall script (**Confirmed** [3]).

Scripts that run on install

- With `ignore-scripts` set, npm does not run scripts from `package.json` files and also disables `.npm-extension` execution. Explicitly named scripts such as `npm test` still run, without their pre and post scripts (**Confirmed** [98]). The default is `false` in npm 11 (**Confirmed** [117]).
- A direct or transitive git dependency can carry an `.npmrc` that overrides the git executable path, which gives code execution during install even with `--ignore-scripts`. npm 11.10.0 added `--allow-git` to control this; it defaulted to `all`, with `none` recommended (**Confirmed** [75]). npm 12 makes `none` the default for both `allow-git` and `allow-remote` (**Confirmed** [79]).

Native binaries and per-platform packages

- A build failure in an `optionalDependency` does not fail the install, `--omit=optional` skips them, and the `os`, `cpu` and `libc` fields restrict a package to matching platforms (**Confirmed** [101]).
- Popular build tools ship one optional package per platform. In the registry on 3 November 2026, esbuild 0.28.2 lists 26 optional `@esbuild/*` packages and a postinstall script, `@swc/core` 1.16.21 lists 12 and a postinstall script, and sharp 0.35.5 lists 25 `@img/sharp-*` packages and no install script. The per-platform packages checked declare `os` and `cpu`, have no scripts, and carry attestations and two registry signatures (**Confirmed** [102]). Each architecture's package is a separate publish that a defender has to verify on its own (**Inferred** [102]).
- esbuild's install script verifies its binary and, if the optional package is missing, attempts to download it from the registry itself. With both `--ignore-scripts` and `--no-optional`, esbuild cannot work. Copying `node_modules` from macOS into a Linux Docker image does not work either; run `npm ci` on the destination (**Confirmed** [103]).
- The older `prebuild-install || node-gyp rebuild` pattern downloads a prebuilt binary from a GitHub release at install time. That project is deprecated; its maintainers recommend prebuildify with node-gyp-build, which ship binaries inside the tarball and work with install scripts disabled (**Confirmed** [104]).

Part 2: The operational record

One row per incident, report or researcher demonstration. Dates are the event dates the sources give; "advisory date" marks a GitHub Advisory Database publish date.

Date	Event	What happened	Grade	Sources
2018-09-09 to 2018-11-26	event-stream	A new maintainer who took over by social engineering added flatmap-stream to event-stream 3.3.6. The payload targeted the Copay wallet (5.0.2 to 5.1.0) and harvested private keys from accounts holding more than 100 BTC or 1000 BCH. npm was notified on 26 November, removed the packages and took ownership of event-stream. GHSA-mh6f-8j2x-4483 (critical, 2018-11-26) lists event-stream 3.3.6 and every flatmap-stream version	Confirmed	[1] [2]
2021-02-09	Dependency confusion	Internal package names uploaded to public npm, PyPI and RubyGems were detected inside more than 35 organizations; Shopify and PayPal each paid $30,000	Demonstrated	[3]
2021-10-22	ua-parser-js	The maintainer believes someone hijacked his npm account; he noticed when his email was flooded with spam and could only deprecate 0.7.29, 0.8.0 and 1.0.0. CISA said the affected versions could let a remote attacker take control of the system and advised updating to 0.7.30, 0.8.1 or 1.0.1	Confirmed	[4] [5]
2021-11-02	npm publish flaw	GitHub received a bug bounty report of a flaw that allowed unauthorized publishing of any package and patched it within six hours	Confirmed	[58]
2021-11-04	coa	2.0.3, 2.0.4, 2.1.1, 2.1.3, 3.0.1 and 3.1.3 published with malware (GHSA-73qr-pfmq-6rp8, critical; downgrade to 2.0.2). A preinstall script ran `compile.js`, which spawns `compile.bat` on Windows; the versions were live from about 14:12 to 15:24 CET and broke downstream builds	Confirmed	[6] [7]
2021-11-04 (advisory date)	rc	1.2.9, 1.3.9 and 2.3.9 published with malware (GHSA-g2q5-5433-rhrf, critical; downgrade to 1.2.8)	Confirmed	[8]
2021-11-15	GitHub CSO statement	Tied the ua-parser-js, coa and rc takeovers (some takeovers involved accounts without 2FA) to a commitment to require 2FA for top-package maintainers from Q1 2022	Confirmed	[58]
2022-01-05 to 2022-01-08	colors and faker	The faker repository was reverted and faker 6.6.6 published empty (5 January); colors 1.4.1 and 1.4.44-liberty-2 added an infinite loop that caused denial of service (8 January). colors had over 20 million weekly downloads and 18,962 dependents, faker about 2 million weekly downloads (Snyk figures)	Confirmed	[9]

Date	Event	What happened	Grade	Sources
2022-03-07 to 2022-03-16	node-ipc "peacenotwar"	10.1.1 and 10.1.2 contained maintainer-introduced code intended to overwrite files depending on IP geolocation (CVE-2022-23812, GHSA-97m3-w2cp-4xx6, advisory published 2022-03-16; removed in 10.1.3). Snyk: 10.1.1 (7 March) wrote a heart emoji over files on hosts in Russia or Belarus; 9.2.2 and 11.x pulled in peacenotwar; about 1.05 million weekly downloads; Vue CLI and Unity affected downstream	Confirmed	[10] [11]
2023-07-18	Jade Sleet / TraderTraitor	GitHub assessed with high confidence that a social engineering campaign was associated with Jade Sleet/TraderTraitor. Personas, some fake and some taken-over accounts, invited targets to repositories that depended on malicious npm packages. GitHub suspended the accounts; no GitHub or npm systems were compromised	Confirmed	[12]
2024-10-30	lottie-player	A user reported that lottie-player CDN builds (2.0.5 named) opened unexpected popups on their site; later comments in the thread describe wallet-connect prompts. LottieFiles attributed 2.0.5 to 2.0.7 to a compromised developer access token, known here only from press coverage of its statement on X	Confirmed (report); Reported (cause)	[13] [14]
2024-12-19	Rspack	`@rspack/core` and `@rspack/cli` 1.1.7 were released by an attacker with malicious scripts. The team invalidated all npm and GitHub tokens, deprecated 1.1.7 and shipped 1.1.8 as a re-release of 1.1.6. The reporter's environment shows `^1.1.6` resolving to 1.1.7	Confirmed	[15]
2025-06-06 to 2025-06-07	Gluestack / React Native Aria	17 packages (16 `@react-native-aria` plus `@gluestack-ui/utils`), over 1 million weekly downloads combined, compromised in waves from 21:33 GMT on 6 June. A remote access trojan hidden by whitespace padding in `lib/commonjs/index.js`, with Windows persistence. The source does not say how publish access was obtained	Confirmed	[16]
2025-07-14 to 2025-10-10	Contagious Interview	Socket identified more than 338 malicious npm packages with over 50,000 downloads, using more than 180 fake personas, since its 14 July update. HexEval and XORIndex loaders rebuild BeaverTail, which then typically fetches InvisibleFerret	Confirmed	[24]
2025-07-18	eslint-config-prettier	Users spotted new versions with no matching commits and no provenance. The maintainer said a phishing email impersonating npm led to his token being stolen	Confirmed	[17]
2025-08-26	Nx "s1ngularity"	A `pull_request_target` workflow echoed an unsanitized PR title, letting the attacker take a GitHub token, trigger the publish workflow and exfiltrate `NPM_TOKEN`. 19 package-version pairs (counted from the post-mortem's version list) were live about 4 hours; Nx detected them because they lacked provenance. The postinstall payload attempted to use local AI CLIs (Claude, Gemini) to search for secrets and posted results to `s1ngularity-repository` repositories. Nx Console 18.6.30 to 18.65.1 installed `nx@latest` when an editor opened, triggering the payload	Confirmed	[18] [19]

Date	Event	What happened	Grade	Sources
2025-09-08	chalk, debug and 16 others	The maintainer was phished by a 2FA-reset email from `support at npmjs dot help` and listed 18 affected packages, including debug 4.4.2, chalk 5.6.1, ansi-styles 6.2.2 and strip-ansi 7.1.1. npm took all impacted versions down (21:59 CEST); his account was restored at 23:48 CEST. Aikido flagged the packages at 13:16 UTC; together they had more than 2 billion weekly downloads. The browser payload hooked `fetch`, `XMLHttpRequest` and `window.ethereum` to swap wallet addresses and transaction recipients. `npmjs.help` was registered on 5 September	Confirmed	[20] [21]
2025-09-14 to 2025-09-23	Shai-Hulud	`@ctrl/tinycolor` 4.1.1 and 4.1.2 (over 2 million weekly downloads) carried `bundle.js`, which ran TruffleHog, dumped environment secrets, pushed a `shai-hulud-workflow.yml` workflow, exfiltrated to webhook.site and a public repository named Shai-Hulud, and republished up to 20 of the maintainer's packages. GitHub was notified on 14 September and removed 500+ packages; CISA called it a self-replicating worm that compromised more than 500 packages. Unit 42 assessed with moderate confidence that an LLM generated the bash script	Confirmed	[22] [70] [23] [27]
2025-10-24 to 2025-11-12	tea.xyz token farming	Amazon Inspector flagged over 150,000 npm packages in a coordinated token-farming campaign, using a rule deployed on 24 October; OpenSSF assigned MAL-IDs through 12 November. AWS notes the packages do not contain overtly malicious code	Confirmed	[25]
2025-11-21 to 2025-11-27	Shai-Hulud 2.0 ("Sha1-Hulud")	Trojanized uploads on 21 to 23 November; leaked-secret repositories from 01:22 UTC on 24 November. The worm runs as a preinstall script (`setup_bun.js`, `bun_environment.js`), registers a self-hosted runner named `SHA1HULUD` and plants workflows. About 700 to nearly 800 packages and 25,000+ repositories by Wiz's count, including Zapier, ENS, PostHog and Postman packages. When it cannot steal or exfiltrate, it tries to destroy the home directory; its repositories are described as "Sha1-Hulud: The Second Coming". CERT-FR dates the wave from 23 November with more than 700 packages by 26 November. At PostHog, an attacker took a bot PAT on 18 November through a `pull_request_target` workflow and used it to take CI secrets, including an npm token, days before the 24 November attack	Confirmed	[26] [27] [28] [29] [30]
2025-11-25	Shai-Hulud 2.0 in Maven Central	The payload appeared in the npm-mirror artifact `org.mvnpm:posthog-node:4.18.1`, which Maven Central purged	Reported	[31]

Date	Event	What happened	Grade	Sources
2026-03-30 to 2026-03-31	axios	Social engineering plus a RAT on the lead maintainer's PC exposed the npm credentials. `plain-crypto-js` 4.2.0 was published at 05:57 UTC on 30 March; axios 1.14.1 at 00:21 UTC on 31 March and 0.30.4 around 01:00; both removed at 03:15 UTC. The attacker deleted community issues. GitHub published malware advisories GHSA-fw8c-xr5c-95f9 and GHSA-2x9r-6wxq-hrr7 the same morning. Socket: 100 million weekly downloads, and no evidence "at this time" linking it to the TeamPCP campaigns	Confirmed	[35] [33] [34] [36]
2026-04-29 to 2026-04-30	Mini Shai-Hulud (SAP CAP)	mbt 1.2.48 (09:55 UTC), `@cap-js/sqlite` 2.2.2 (11:25), `@cap-js/db-service` 2.10.1 and `@cap-js/postgres` 2.2.2 (12:14), each detected within 6 to 8 minutes. A preinstall Bun payload steals cloud and CI credentials into repositories described as "A Mini Shai-Hulud has Appeared". SAP issued Security Note 3747787 on 30 April (as relayed by Onapsis). Malware advisories on 30 April cover `@cap-js/sqlite`, `@cap-js/db-service` and intercom-client 7.0.4. Kudelski reports spread to PyPI lightning 2.6.2 and 2.6.3 and says the campaign has been attributed to TeamPCP, without saying by whom	Confirmed; Reported (attribution)	[38] [39] [40] [41] [42]
2026-05-11	TanStack	84 malicious versions across 42 `@tanstack` packages, published between 19:20 and 19:26 UTC through `pull_request_target`, GitHub Actions cache poisoning and an OIDC token read from runner memory, authenticated through the project's trusted-publisher binding. Detected 20 to 26 minutes after publish, depending on the batch, and deprecated within the hour	Confirmed	[43]
2026-05-14	node-ipc (2026)	9.1.6, 9.2.3 and 12.0.1 published from account `atiertant`, which had no prior publish history on the package, about 21 months after the last legitimate release. The obfuscated credential stealer runs on `require()`, not at install. Over 10 million weekly downloads (StepSecurity figure). GHSA-pvh2-rg5g-69v7 (advisory date 2026-05-29) lists 9.1.6 as malware	Confirmed	[44] [45]
2026-05-29 to 2026-06-01	Red Hat Cloud Services	A GitHub account compromised through a malicious VS Code extension was used with the Miasma ("Mini Shai-Hulud") kit; 32 `@redhat-cloud-services` packages were compromised. Red Hat verified that no product builds contained the compromised versions	Confirmed	[46]
2026-08-04 to 2026-08-06	keyv and the CHAINDROP worm	The keyv maintainer's GitHub account was compromised on 4 August. By 5 August at least 444 packages (1,381 versions) were affected (Aikido). The releases had valid provenance, the fallback C2 was resolved from an Ethereum contract, and exfiltration repositories were described as "Shai-Hulud: Here We Go Again". Elastic named the worm CHAINDROP: it propagates only with an npm token that has write access and `bypass_2fa`; keyv alone had over 600 million downloads in the prior month. Malware advisories cover keyv 6.0.0, flat-cache 6.1.24 and cache-manager 7.2.10	Confirmed	[47] [48] [49] [50] [51]

Date	Event	What happened	Grade	Sources
2026-10-05 and 2026-10-08 (advisory dates)	`@subql/common`, tensorlake	Malware advisories for `@subql/common` 5.8.3 and tensorlake 0.5.144 (the tensorlake advisory also carries a `>= 0` range). The sources establish only that these versions are malware	Confirmed	[53] [54]

2018 to 2022: one maintainer at a time

The early incidents each turned on one maintainer. event-stream was taken over through social engineering by an attacker posing as a maintainer [1]. ua-parser-js, coa and rc were account takeovers, and GitHub's security chief noted that such takeovers at times involved accounts without 2FA [58]. The coa versions were on the registry for about an hour and still broke builds in svgr, Vue CLI, react-scripts, Nuxt and CI pipelines, which shows how fast a popular release fans out [7]. The 2022 cases were different in kind: the maintainers of colors, faker and node-ipc changed their own packages on purpose [9] [10]. Account security does not address that case; pinning, review and release-age gates do (**Inferred**).

2023 to 2024: fake recruiters and CDN builds

GitHub's July 2023 alert described a state-linked group using npm as a lure inside repositories it invited targets to [12]. The lottie-player case in October 2024 reached users through CDN builds, not only through npm installs: the reporter named the unpkg `@latest` build and the jsDelivr 2.0.5 build, and the popup opened on the reporter's own site [13]. Rspack in December 2024 is the first case in this record where the affected project invalidated all its existing npm and GitHub tokens [15] (**Inferred** that it is the first).

2025: the phishing summer and the worms

Between June and September 2025 the record moves from single packages to groups of packages owned by one maintainer (**Inferred**). The eslint-config-prettier and chalk/debug maintainers were both phished by emails that impersonated npm [17] [20]. The chalk/debug wave is the largest by weekly downloads in this record (**Inferred** from [21] [36] [48]), and its payload ran in the browser, swapping wallet addresses inside bundled web applications [21]. Nx showed a new access path, a poisoned GitHub Actions workflow, and a new payload behavior: asking the developer's own AI coding tools to look for secrets [18].

Shai-Hulud in September made the step from compromise to worm: it used each stolen token to republish the victim's other packages [22]. The November "Second Coming" wave moved the payload to preinstall, registered infected machines as self-hosted GitHub runners and added a destructive fallback [26] [27] [28]. PostHog's post-mortem shows the worm's reach depended on tokens stolen earlier through CI: the attacker took a bot PAT on 18 November and used it to collect CI secrets, including the npm token, days before the 24 November attack [30].

2026: trusted pipelines, editors and the long tail

In 2026 the attackers went through the controls the registry had added. TanStack's malicious versions were published through its OIDC trusted-publisher binding [43], and the keyv releases carried valid provenance [47]. axios was compromised through malware on the lead maintainer's computer [35], and Red Hat's packages through a malicious VS Code extension [46]. The node-ipc stealer of May 2026 avoided install scripts entirely and ran on `require()` [44]. The CHAINDROP worm in August spread only through tokens set to bypass 2FA, which GitHub had restricted for account changes on 31 July but which could still publish directly until the change targeted for January 2027 [48] [84] (**Inferred** from the two dates).

Part 3: How attackers got publish access

Path	Cases	Grade	Sources
Social engineering of a maintainer handover	event-stream (2018)	Confirmed	[1]
Account hijack (GitHub: takeovers at times involved accounts without 2FA)	ua-parser-js, coa, rc (2021)	Confirmed	[5] [58]
The maintainer, on purpose	colors, faker (2022); node-ipc 10.1.x (2022)	Confirmed	[9] [10]
Compromised developer access token	lottie-player (2024)	Reported	[14]
Phishing that impersonated npm	eslint-config-prettier (2025-07); chalk, debug and 16 others (2025-09), via `npmjs.help`	Confirmed	[17] [20] [21]
CI workflow injection under `pull_request_target`	Nx (2025-08), PostHog (2025-11), TanStack (2026-05, with cache poisoning and an OIDC token read from runner memory)	Confirmed	[18] [30] [43]
Tokens the worm stole from infected machines, reused to republish the victim's other packages	Shai-Hulud (2025-09), Shai-Hulud 2.0 (2025-11), Mini Shai-Hulud (2026-04), CHAINDROP (2026-08)	Confirmed	[22] [26] [161] [48]
Malware on the maintainer's own machine	axios (2026-03, social engineering plus a RAT); Red Hat (2026-05, a malicious VS Code extension)	Confirmed	[35] [46]
Compromised GitHub account	keyv (2026-08)	Confirmed	[47]
Not stated by the source	Rspack (2024), Gluestack (2025), node-ipc 2026 (account `atiertant`, compromised or newly added)	Confirmed (that the path is unknown)	[15] [16] [44]
Packages created by the attacker	Contagious Interview personas (2025); tea.xyz token farming (2025)	Confirmed	[24] [25]

Ultralytics, on PyPI in December 2024, is the closest cross-registry parallel: the project used Trusted Publishing, the first malicious releases came through GitHub Actions cache poisoning, and a second round came through an unrevoked API token that was detectable by its missing publish attestations (**Confirmed** [55]). The TanStack incident 17 months later used the same cache-poisoning step against npm [43]. PyPI's Trusted Publishing exchanges a CI OIDC token for an API token that expires after 15 minutes, and its PEP 740 attestations carry PyPI Publish and SLSA Provenance predicates (**Confirmed** [156]); as with npm, a

short-lived token does not help once attacker code runs inside the release job (**Inferred** [55] [43]).

Part 4: How the payloads ran

Execution point	Cases	Grade	Sources
preinstall script	coa (2021); Shai-Hulud 2.0 (2025-11); Mini Shai-Hulud (2026-04); CHAINDROP (2026-08)	Confirmed	[7] [28] [38] [48]
Runs during `npm install` through a bundled script	Shai-Hulud `bundle.js` (2025-09); the Nx postinstall (2025-08)	Confirmed	[22] [19]
Even when installation fails	Shai-Hulud 2.0 preinstall	Confirmed	[28]
On `require()` (load time, no install script)	node-ipc 2026; the Gluestack RAT in `lib/commonjs/index.js` (Inferred: the source names the file, not the trigger)	Confirmed; Inferred	[44] [16]
In the browser, from bundled code or a CDN build	chalk/debug clipper (2025-09); lottie-player popups (2024-10)	Confirmed	[21] [13]
Through an editor or AI tool	Nx Console installed `nx@latest` on editor open; the Nx payload attempted to use Claude and Gemini CLIs; CHAINDROP plants a `.claude/settings.json` SessionStart hook and a `.vscode/tasks.json` folderOpen task	Confirmed	[19] [18] [48]
Inside CI with the project's own credentials	Shai-Hulud workflow push; Shai-Hulud 2.0 self-hosted runner `SHA1HULUD`; TanStack publish through its trusted-publisher binding	Confirmed	[22] [26] [43]

Most waves executed through lifecycle scripts, which npm v12's default-off `allowScripts` addresses. The 2026 node-ipc stealer and the chalk/debug browser clipper would not be stopped by that default, because `allowScripts` governs only install-time scripts (**Inferred** [78]). The Nx advisory makes the same point from the other side: packages get installed through transitive dependencies, AI agents, editors and extensions, not only through a developer typing `npm install` (**Confirmed** [19]).

Part 5: The registry and package-manager response

npm and GitHub, 2019 to 2026

Date	Change	Grade	Sources
2019-05-23	Dependabot automated security fixes, public beta	Confirmed	[56]
2021-10-03	The npm advisory database became part of the GitHub Advisory Database; `npm audit` links to GitHub advisories	Confirmed	[57]
2021-12-07 to 2022-01-04	Enhanced login verification (an emailed one-time code) for publishers without 2FA	Confirmed	[59]
2022-02-01	2FA mandatory for maintainers of the top-100 packages by dependents; organization-level 2FA enforcement	Confirmed	[60]
2022-03-01	All npm accounts enrolled in login verification (a baseline, not a replacement for 2FA)	Confirmed	[61]
2022-05-31	2FA enforced for maintainers of the top-500 packages by dependents	Confirmed	[62]
2022-06-15	Malware advisories (`type:malware`) in the GitHub Advisory Database; Dependabot opens no update PRs for malware	Confirmed	[63]
2022-07-26	Registry moved from PGP to ECDSA signatures with HSM-managed keys; `npm audit signatures` announced for npm 8.13.0 [64] (npm's current documentation requires 8.15.0 or later [114])	Confirmed	[64]
2022-11-01	2FA required for high-impact packages (more than 1 million weekly downloads or more than 500 dependents)	Confirmed	[65]
2023-03-21	Granular access tokens GA: limited to packages, scopes or organizations, with expiry, IP ranges and read or write	Confirmed	[66]
2023-04-19	Package provenance public beta: Sigstore Fulcio certificate from the CI OIDC token, Rekor log entry, `--provenance` from GitHub Actions, verified by `npm audit signatures` (npm 9.5.0 and later)	Confirmed	[67]
2023-09-26	Provenance GA for GitHub Actions and GitLab CI/CD, public packages from public repositories only	Confirmed	[68]
2025-07-31	Trusted publishing with OIDC GA for GitHub Actions and GitLab CI; npm CLI 11.5.1 or later; provenance by default	Confirmed	[69]
2025-09-22	GitHub's plan after Shai-Hulud: local publishing with 2FA, granular tokens with a seven-day lifetime, or trusted publishing; deprecate classic tokens and TOTP 2FA in favor of FIDO; tokens disallowed by default. The plan gave no dates	Confirmed	[70]
2025-09-29	New write tokens default to 7 days (maximum 90); new TOTP setups disabled; classic tokens to be revoked in mid-November	Confirmed	[71]
2025-11-05	Classic token creation disabled; existing write granular tokens capped at 90 days; new write tokens enforce 2FA with Bypass 2FA off by default; revocation scheduled for 19 November	Confirmed	[72]

Date	Change	Grade	Sources
2025-12-09	All classic tokens permanently revoked; `npm login` issues two-hour session tokens with 2FA on publish; `npm token create` arrives; new packages have 2FA on by default	Confirmed	[73]
2026-02-11	npm 11.10.0 adds `min-release-age` (days); 11.17.0 (2026-06-11) adds `min-release-age-exclude`	Confirmed	[74]
2026-02-18	Bulk trusted-publishing configuration through `npm trust`; `--allow-git` for git dependencies (npm 11.10.0 and later)	Confirmed	[75]
2026-03-17	Dependabot malware alerts for npm return as an opt-in feature; the 2022 pause had been caused by false positives from public and private packages sharing names	Confirmed	[76]
2026-05-22	Staged publishing GA: `npm stage publish` queues the tarball for a human maintainer's 2FA approval; trusted publishing configurations can be stage-only (npm 11.15.0 or later); `--allow-file`, `--allow-remote` and `--allow-directory` added	Confirmed	[77]
2026-06-09	npm v12 preview: `allowScripts` off by default, including implicit node-gyp builds and prepare scripts of git, file and link dependencies; `--allow-git` and `--allow-remote` default to `none`; npm 11.16.0 and later warn	Confirmed	[78]
2026-07-08	npm 12.0.0 released (21:21:30 UTC) and tagged `latest`: dependency lifecycle scripts blocked unless the root `allowScripts` policy allows them; `npm-shrinkwrap.json` no longer honored; unknown CLI flags throw	Confirmed	[79] [81]
2026-07-28	Publish-time malware scanning before a new version can be installed (typically about five minutes, 15 or more at peak); dual-use packages declare `contentPolicy` and ship a `DISCLOSURE` file, enforced progressively. Dependabot malware alerts extend beyond npm using OpenSSF malicious-packages data	Confirmed	[82] [83]
2026-07-31	Bypass-2FA tokens can no longer change tokens, package access, maintainers, trusted publishing or org membership without an interactive 2FA challenge; removing their direct publishing is targeted for January 2027	Confirmed	[84]
2026-09-03	Multiple trusted publishing configurations per package; staging by default, direct publishing opt-in; approval disabled until the malware scan finishes	Confirmed	[85]
2026-09-18	"Read and write (stage only)" granular tokens: `npm publish` is rejected even with bypass-2FA, but the token can still move dist-tags and deprecate versions	Confirmed	[86]
2026-10-02	Unvalidated trusted publishing configurations expire after 48 hours; OIDC tokens from `issue_comment` events rejected, as `pull_request_target` tokens already were; staged publishing can create new packages	Confirmed	[87] [88]

On 3 November 2026 the current releases are npm 12.2.1, pnpm 12.16.1, Yarn classic 1.22.22, Yarn Berry 4.18.1, bun 1.4.4, osv-scanner v2.7.0, syft v1.56.0, grype v0.121.1, nvm v0.40.8 and volta v2.0.2 (**Confirmed** [97]).

pnpm and Yarn

Date	Release	Change	Sources
2025-01-07	pnpm 10.0.0	Dependency lifecycle scripts not run by default; allow through `onlyBuiltDependencies`	[89]
2025-04-09	Yarn 4.9.0	npm provenance support	[94]
2025-09-12	pnpm 10.16.0	`minimumReleaseAge` (minutes) and `minimumReleaseAgeExclude`	[90]
2025-09-18	Yarn 4.10.0	`npmMinimalAgeGate`, `npmPreapprovedPackages`, OIDC auth for GitHub Actions and GitLab	[95]
2025-11-09	pnpm 10.21.0	`trustPolicy: no-downgrade` fails an install when a package's trust level drops, for example from trusted publisher to provenance only	[91]
2025-12-15	pnpm 10.26.0	`blockExoticSubdeps` (transitive dependencies must resolve from trusted sources) and the `allowBuilds` map	[92]
2026-04-28	pnpm 11.0.0	Supply-chain protection on by default: `minimumReleaseAge` of 1 day, `blockExoticSubdeps` and `strictDepBuilds` true; old build settings removed in favor of `allowBuilds`	[93]
2026-06-02	Yarn 4.16.0	`yarn npm stage`	[96]

All rows **Confirmed** from each project's release page.

Part 6: The vulnerability-management view

What known-vulnerability tooling sees

`npm audit`, OWASP Dependency-Check and grype match installed versions against advisory data. A malicious version published an hour ago has no CVE (**Inferred**). node-ipc's 2022 protestware received a CVE id (CVE-2022-23812) [10]; most other incidents here appear as GitHub malware advisories, and the coa and node-ipc 9.1.6 advisories carry CWE-506 Embedded Malicious Code [6] [45]. Those advisories come from the npm security team and the OpenSSF Malicious Packages repository, and Dependabot alerts on them only when malware alerts are enabled (**Confirmed** [149]). Dependabot does not open update PRs for malware, because the only fix is to remove the package (**Confirmed** [63]). As of 3 November 2026 GitHub lists malware alert coverage for Composer, Go, Maven, npm, NuGet, pip, RubyGems and Rust (**Confirmed** [148]).

Exposure windows

Case	Window	Source
coa (2021)	About 14:12 to 15:24 CET, about 72 minutes	[7]
Nx (2025)	About 4 hours	[18]
axios 1.14.1 (2026)	00:21 to 03:15 UTC, under 3 hours	[35]
Mini Shai-Hulud (2026)	Detected 6 to 8 minutes after each publish	[38]
TanStack (2026)	Detected 20 to 26 minutes after publish, depending on the batch, and deprecated within the hour	[43]
npm publish-time scan (from July 2026)	New versions held typically about five minutes, 15 or more at peak, before install	[82]

pnpm's 10.16.0 release note puts it plainly: such attacks are usually discovered quickly and removed within an hour (**Confirmed** [90]). A release-age gate of a day or more would have skipped every version in the table above (**Inferred**). A gate also delays urgent security fixes (**Inferred**); npm, pnpm and Yarn each offer an exclude or preapproval list for that case (`min-release-age-exclude`, `minimumReleaseAgeExclude`, `npmPreapprovedPackages`) [74] [90] [95].

What provenance does and does not prove

Provenance proved its worth as a tripwire. Nx detected its bad versions because they lacked provenance, and eslint-config-prettier users noticed releases that had no matching commits and no provenance [18] [17]. pnpm's `trustPolicy: no-downgrade` turns that tripwire into an install failure [91]. But Nx had provenance on its recent releases before the attack, and that did not stop users installing the malicious versions [19]. TanStack's malicious versions were published through the project's own OIDC trusted-publisher binding [43], which generates provenance automatically [69] (**Inferred** that they carried it), and the keyv releases had valid provenance signed by GitHub Actions [47]. npm says provenance does not guarantee the package has no malicious code (**Confirmed** [121]). In SLSA terms, provenance at Build L2 is signed by a hosted platform and validated by consumers, and Build L3 adds isolated runs with signing secrets kept away from user steps (**Confirmed** [144]); a pipeline that runs attacker code under `pull_request_target` defeats the point of either (**Inferred** [43] [18]). npm's documentation does not state a SLSA level for its provenance (**Confirmed** [121]).

Indicator lists to check against

- Wiz publishes `shai-hulud-2-packages.csv` (a `Package,Version` header, versions written like `= 0.0.7 || = 0.0.8`, 795 data rows, committed 24 to 27 November 2025) and `keyv-packages.csv` (446 entries for the August 2026 wave, three of them Go module paths) (**Confirmed** [32] [52]).
- Datadog's `check-dependencies-ranges.mjs` (npm, Yarn and pnpm) tests whether a project's declared ranges could have resolved to the bad axios versions; the repository warns that a clean result is not a guarantee (**Confirmed** [37]).
- Elastic's CHAINDROP report lists `setup.mjs`, `Math_Symbol.js` and `math_init.js` with SHA-256 values, the preinstall hook, the `.claude/settings.json` SessionStart hook, the `.vscode/tasks.json` folderOpen task and "chore: update config" commits from `claude@users.noreply.github.com` (**Confirmed** [48]).

Part 7: The defender view

Step 1: find every Node tree and install root

```
# Every package-lock.json on the root filesystem, without descending into node_modules [137] [
138]
find / -xdev -name node_modules -prune -o -name package-lock.json -print

# The same walk for all five lockfile names at once (the study's composition of the documented
flags)
find / -xdev -name node_modules -prune -o \( -name package-lock.json -o -name npm-shrinkwrap.
json \
  -o -name pnpm-lock.yaml -o -name yarn.lock -o -name bun.lock \) -print

# Global install roots [109] [125] [128]
npm ls --global          # lists {prefix}/lib/node_modules
pnpm root -g             # prints the global packages directory
yarn global dir          # Yarn classic; default ~/.config/yarn/global
```

GNU and FreeBSD `find` both document `-prune` as not descending into a directory, and GNU documents `-xdev` as not crossing onto other devices (**Confirmed** [137] [138]); the grouped form is **Inferred**. nvm installs to `~/.nvm` when `NVM_DIR` is unset (or `$XDG_CONFIG_HOME/nvm` when that is set), keeps each version's globals under `~/.nvm/versions/node/vX/lib/node_modules`, and runs a command against one version with `nvm exec <ver> <cmd>`; nvm-windows is a separate project (**Confirmed** [136]). Each Node version an nvm user has installed is a separate global root to check (**Inferred** [136]).

On Windows, the study's PowerShell equivalent of the lockfile walk is below (**Inferred**, not run for this study):

```
Get-ChildItem -Path C:\ -Recurse -Include package-lock.json,pnpm-lock.yaml,yarn.lock,bun.lock -
ErrorAction SilentlyContinue |
  Where-Object { $_.FullName -notmatch '\\node_modules\\' } | Select-Object -ExpandProperty
FullName
```

Step 2: check for a known-bad version

```
# npm: is axios 1.14.1 anywhere in this tree? [109] [110] [111]
npm ls axios@1.14.1 --all
npm query "#axios@1.14.1" --expect-result-count=0   # non-zero exit if it is present
npm explain axios                                    # alias: npm why; prints the chain that
pulled it in
npm ls --all --package-lock-only                     # work from the lockfile rather than
node_modules

# Packages that declare a postinstall script [110]
npm query ":attr(scripts, [postinstall])"

# pnpm [125]
pnpm ls "axios@1.14.1" -r --depth Infinity --json
pnpm why axios -r --json

# Yarn classic and Yarn Berry [128] [129]
yarn why axios
yarn list --pattern axios
yarn why axios@1.14.1 -R
```

All flags above are documented by their tools (**Confirmed** [109] [110] [111] [125] [128] [129]). For a wave with hundreds of packages, compare `npm ls --all --json` or each lockfile against the Wiz CSV for that wave [32] [52], and for axios run Datadog's range checker over each project [37]. CISA's Shai-Hulud guidance adds

two places people forget: nested dependencies inside lockfiles, and cached copies in artifact repositories and dependency-management tools (**Confirmed** [23]).

Step 3: audit and verify

```
# Known vulnerabilities; needs a lockfile; exits 0 when there are none [112]
npm audit --audit-level=high

# Registry signatures and provenance attestations; run after npm install or npm ci (npm 8.15.0+)
[112] [114]
npm audit signatures
npm audit signatures --json --include-attestations    # adds a "verified" array with the
Sigstore bundles

# pnpm: exits 1 while an advisory at or above the level remains [124]
pnpm audit --audit-level high --prod
pnpm audit signatures

# Yarn classic exit code is a severity bitmask: 1 info, 2 low, 4 moderate, 8 high, 16 critical [
128]
yarn audit
# Yarn Berry [129]
yarn npm audit -A -R --severity high --json

# bun: reads bun.lock; exits 1 when vulnerabilities remain or the registry request fails [131]
bun audit --audit-level=high --prod

# grype: exit code 2 at or above the severity [132]
grype dir:. --fail-on high -o json
grype db update

# OWASP Dependency-Check (an NVD API key is strongly recommended) [134]
dependency-check.sh --out . --scan .

# An SBOM per build [120] [135]
npm sbom --sbom-format cyclonedx --sbom-type application
cyclonedx-npm --output-file sbom.cdx.json
```

Every command and flag above is documented by its tool (**Confirmed**). `npm audit --audit-level` changes the failure threshold without filtering the report [112]. `npm audit signatures` errors when packages lack signatures while the registry offers signing keys, which npm says could mean an attacker is trying to get around verification [114]; the npm page documents no exit codes for it [113]. Yarn classic's `--level` flag does not change its exit code [128]. OWASP Dependency-Check is an OWASP Flagship tool that matches dependencies to CPE and CVE using NVD data [133]; its Node analyzers read `package.json`, `package-lock.json` or `npm-shrinkwrap.json` (Node Audit, which needs Internet access), `pnpm-lock.yaml` and `yarn.lock` [134]. In GitHub, `actions/dependency-review-action` fails pull requests that add vulnerable dependencies, with `fail-on-severity`, `allow-ghsas`, `comment-summary-in-pr`, `show-openssf-scorecard` and `warn-only` options, on public repositories and private repositories with GitHub Advanced Security (**Confirmed** [150]). The dependency graph exports an SPDX SBOM from Insights or the REST API (**Confirmed** [147]).

The same idea exists in other ecosystems a Node shop often also runs: `bundle-audit check --update` for Ruby [157], `cargo audit` and `cargo audit bin` for Rust (run it with `--file` on untrusted projects, because without a lockfile it runs `cargo update`) [158], and `dotnet list package --vulnerable --include-transitive` plus NuGetAudit on restore for .NET, with NU1901 to NU1904 warnings that can be raised to errors [159] (**Confirmed**).

Step 4: remove and reinstall cleanly

```
# Remove a package from dependencies, devDependencies, optionalDependencies and peerDependencies,
# and update the lockfile if one exists [115]
npm uninstall <pkg>
npm uninstall -g <pkg>      # --no-save is ignored in global mode

# Reinstall exactly the lockfile: deletes node_modules first, never writes package.json or the
lockfile [117]
npm ci --ignore-scripts

# Cache [118]
npm cache verify
npm cache clean --force

# Yarn Berry: clear both caches, then refetch and re-verify checksums [129]
yarn cache clean --all
yarn install --immutable --check-cache
```

To force a known-good version anywhere in the tree, add `overrides` to the root `package.json` (npm reads overrides only from the root; they can be nested under a parent, can use `$` references to a direct dependency, and fail with `EOVERRIDE` when they conflict with a direct dependency spec) [116]:

```
{
  "overrides": {
    "<pkg>": "<known-good-version>"
  }
}
```

pnpm overrides go in the root `pnpm-workspace.yaml` and support `parent>target` scoping, and `pnpm audit --fix` writes overrides there [126] [124]. Yarn classic uses `resolutions` such as `"d2/left-pad": "1.1.1"`, and Yarn Berry reads `resolutions` from the root only [128] [129] (**Confirmed**).

On the cache: npm describes its cache as content-addressable, integrity-checked and self-healing, so clearing it is mainly for reclaiming space [118]. Wiz's Shai-Hulud 2.0 guidance still says to run `npm cache clean --force` and remove `node_modules` [26]. The two are consistent: the cache is faithful, so it will serve a malicious tarball whose integrity matches a poisoned lockfile, and cleaning removes that local copy (**Inferred** [118] [26]).

Step 5: rotate, revoke and hunt for worm artifacts

```
# npm tokens [119]
npm token revoke <id-or-unique-prefix>
npm token create --expires 30 --packages <pkg> --cidr <range> \
  --packages-and-scopes-permission read-write-stage-only

# Repositories the worms create in the victim's own account [155]
gh search repos "Sha1-Hulud" --match description --owner <account> --json name,description,
createdAt
gh search repos "Shai-Hulud" --match name --owner <account> --json name,description,createdAt

# Workflow and commit artifacts (the study's composition; Inferred, not run for this study)
git log --all --oneline -- .github/workflows/shai-hulud-workflow.yml .github/workflows/
discussion.yaml
git log --all --oneline --author=claude@users.noreply.github.com --grep="chore: update config"
```

- `gh auth logout` removes only local credentials and does not revoke the token; to revoke every token the GitHub CLI issued, revoke "GitHub CLI" at github.com/settings/applications (**Confirmed** [151]).

- Account settings can revoke all GitHub credentials at once, revoking an OAuth or GitHub App authorization revokes its tokens, and tokens pushed to public repositories or gists are revoked automatically (**Confirmed** [152]).
- `POST /credentials/revoke` revokes exposed GitHub tokens you do not own (`ghp_`, `github_pat_`, `gho_`, `ghu_`, `ghr_`). It accepts only unauthenticated requests, up to 1,000 tokens per request and 60 requests per hour (**Confirmed** [153]).
- An npm trusted publishing connection cannot be edited: delete it under the package's Settings and create a new one (**Confirmed** [122]).
- OIDC trust between GitHub Actions and a cloud provider is configured on the cloud side and matches token claims (`sub`, `repository`, `ref`, `environment`, `job_workflow_ref`). Tightening those conditions limits it; deleting the trust definition to revoke it is the study's inference, since GitHub's page does not describe revocation (**Confirmed**; **Inferred** for revocation [154]).
- The artifact names in the `git log` lines come from ATT&CK's S9008 procedures (`.github/workflows/discussion.yaml`) and the StepSecurity and Elastic reports (`shai-hulud-workflow.yml`, the CHAINDROP commit author and message) (**Confirmed** [160] [22] [48]). Also review each repository's and organization's self-hosted runners for one named `SHA1HULUD` [26], and each developer machine for the `.claude/settings.json` SessionStart hook and `.vscode/tasks.json` folderOpen task CHAINDROP plants [48].
- CISA's list for the first wave: rotate all developer credentials, require phishing-resistant MFA on GitHub and npm, block outbound connections to webhook.site, remove unneeded GitHub Apps and OAuth apps, audit webhooks and secrets, and turn on branch protection, secret scanning and Dependabot (**Confirmed** [23]). Elastic adds revoking npm tokens, especially `bypass_2fa` automation tokens, and upgrading to npm 12 (**Confirmed** [48]). CERT-FR adds a temporary freeze on npm updates where possible (**Confirmed** [29]).

Step 6: harden

Install time

Control	npm	pnpm	Yarn Berry	Grade	Sources
Dependency install scripts	Off by default in npm 12; approve with `npm install-scripts approve`, then run them with `npm rebuild`; `strict-allow-scripts` makes unreviewed scripts a hard error; `dangerously-allow-all-scripts` is a migration escape hatch. npm 11: `ignore-scripts`	Blocked unless listed in `allowBuilds`; `strictDepBuilds` (default true) fails the install on unreviewed builds; approve with `pnpm approve-builds`	`enableScripts` defaults to false for third-party postinstall scripts	Confirmed	[79] [80] [107] [105] [108] [127] [130]
Release-age gate	`min-release-age` in days (11.10.0+), `min-release-age-exclude` (11.17.0+)	`minimumReleaseAge` in minutes, default 1440 (non-strict), `minimumReleaseAgeExclude`	`npmMinimalAgeGate` (for example `1w`), `npmPreapprovedPackages`	Confirmed	[74] [126] [130] [95]

Control	npm	pnpm	Yarn Berry	Grade	Sources
Exotic sources	`allow-git`, `allow-remote` default `none` in npm 12; `--allow-file`, `--allow-directory`	`blockExoticSubdeps` default true	not covered here	Confirmed	[79] [77] [126]
Lockfile only	`npm ci` errors on a manifest and lock mismatch	not covered here	`enableImmutableInstalls` (default on CI); `checksumBehavior` defaults to `throw`	Confirmed	[117] [130]
Trust downgrade	not covered here	`trustPolicy: no-downgrade` (default off)	`enableHardenedMode` checks the lockfile against the registry, on automatically for PRs from public GitHub repositories	Confirmed	[126] [130]

npm's `allowScripts` matches a dependency's resolved identity, not its self-reported name, approvals are pinned to `pkg@version` by default, and the policy covers preinstall, install and postinstall, plus prepare for non-registry dependencies (**Confirmed** [98] [106]). `npm install-scripts` has `approve`, `deny`, `ls` and `prune` subcommands, stores the policy in the `allowScripts` field of `package.json`, and fails with `--global` (`EGLOBAL`) (**Confirmed** [107]). The June 2026 preview named the commands `approve-scripts` and `deny-scripts`; the v12.0.0 changelog and the shipped v12 documentation use `npm install-scripts approve` [78] [80] [107]. Native packages differ. With scripts off, esbuild's platform binary still installs through its optional package and the CLI runs through a JavaScript shim (**Confirmed** [103]); a `prebuild-install || node-gyp rebuild` package gets no binary unless its script is approved or it moves to prebuildify (**Inferred** [104]).

The study's own verify lines for these settings are below (**Inferred**, not run for this study):

```
npm config get ignore-scripts
npm config get min-release-age
npm config get allow-git
npm --version        # 12.x blocks dependency install scripts by default
```

In CI

- Install with `npm ci` or `npm install-ci-test`, which treat the lockfile as read-only; `npm install`, `npm update`, `npm exec` and `npx` may fetch unpinned dependencies. Run CI with least privilege, no secrets and read-only permissions where possible (**Confirmed** [139]).
- Never check out untrusted code under `pull_request_target`, pass untrusted values through environment variables, avoid self-hosted runners on public repositories, default `GITHUB_TOKEN` to read-only, use OIDC over long-lived secrets, and pin actions to full commit SHAs. GitHub's current page also describes workflow dependency lockfiles as the stronger replacement for SHA pinning where a repository can adopt them (**Confirmed** [145]). Nx, PostHog and TanStack each broke one of these rules (**Inferred** from [18] [30] [43]). PyPI's Ultralytics analysis gives the same advice: avoid `pull_request_target` and shared caches in release workflows, and pin dependencies with hashes [55].
- Keep install scripts away from secrets: OWASP's CICD-SEC-3 recommends running install scripts in a context without access to secrets, an internal proxy, checksum and signature verification, pinning, scoped private packages and a committed `.npmrc` (**Confirmed** [173]).

- GitHub push protection blocks pushes that contain secrets; repository-level push protection requires GitHub Secret Protection, and push protection for users is on by default for public repositories (**Confirmed** [146]).

Publish time

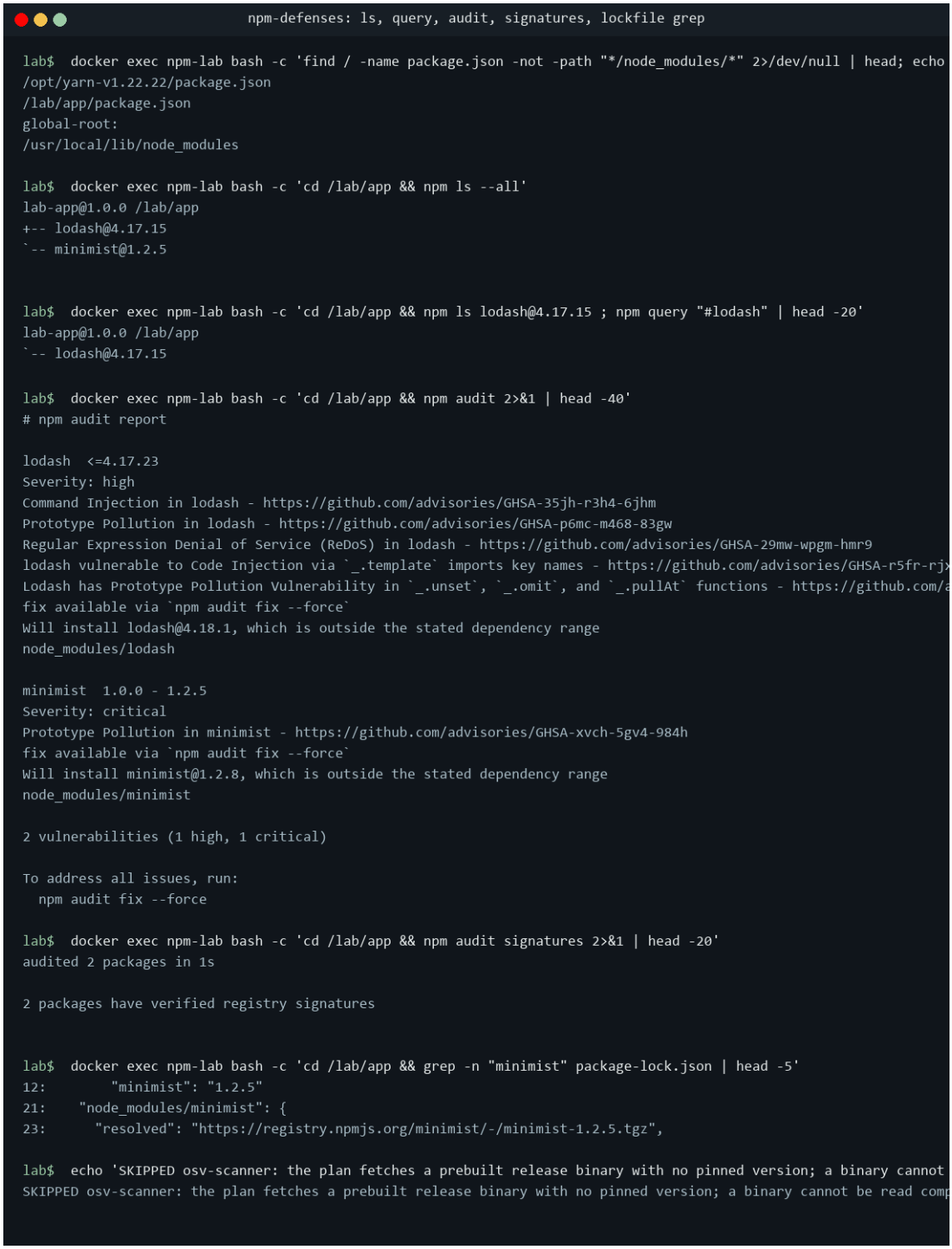
- Trusted publishing over OIDC from GitHub Actions, GitLab CI/CD or CircleCI cloud runners (no self-hosted runners), with npm 11.5.1+ and Node 22.14.0+. It generates provenance automatically for public packages from public GitHub and GitLab repositories. Configurations created after 3 September 2026 default to `npm stage publish`. npm recommends "Require two-factor authentication and disallow tokens" and revoking tokens you no longer need (**Confirmed** [122]).
- Staged publishing puts each release behind a maintainer's 2FA approval, which stays disabled until the publish-time malware scan finishes [77] [85]. Stage-only tokens give automation a publish path that a stolen token cannot use to ship directly [86] (**Confirmed**). A stage-only token can still move dist-tags and deprecate versions, so it is not harmless if stolen [86] (**Inferred** from the same page).
- 2FA with WebAuthn security keys (Touch ID, Face ID, Windows Hello, YubiKey), set to `auth-and-writes`; logging in with a recovery code puts a 72-hour hold on the account (**Confirmed** [123]).
- Any remaining token: `--expires`, `--packages`, `--scopes`, `--cidr`, and the `read-write-stage-only` permission [119].

At the registry edge

- Verdaccio uplinks proxy and cache npmjs; rules use `access`, `publish` and `proxy`, a package without `proxy` stays local, and private scopes must be listed before the catch-all rule (**Confirmed** [140]). That last point is the dependency-confusion defense in configuration form (**Inferred** [140] [3]).
- JFrog Curation offers Malicious package, Package version is immature and Package integrity violation conditions, with npm supported (**Confirmed** [141]).
- Sonatype Firewall quarantines newly requested components that fail a policy at the Proxy stage, fails closed when its service is unavailable, typically returns 403 to the requester, and releases on waiver or automatically (**Confirmed** [142]).
- The GitHub Packages npm registry supports only scoped packages, mapped per scope in `.npmrc` (**Confirmed** [143]).

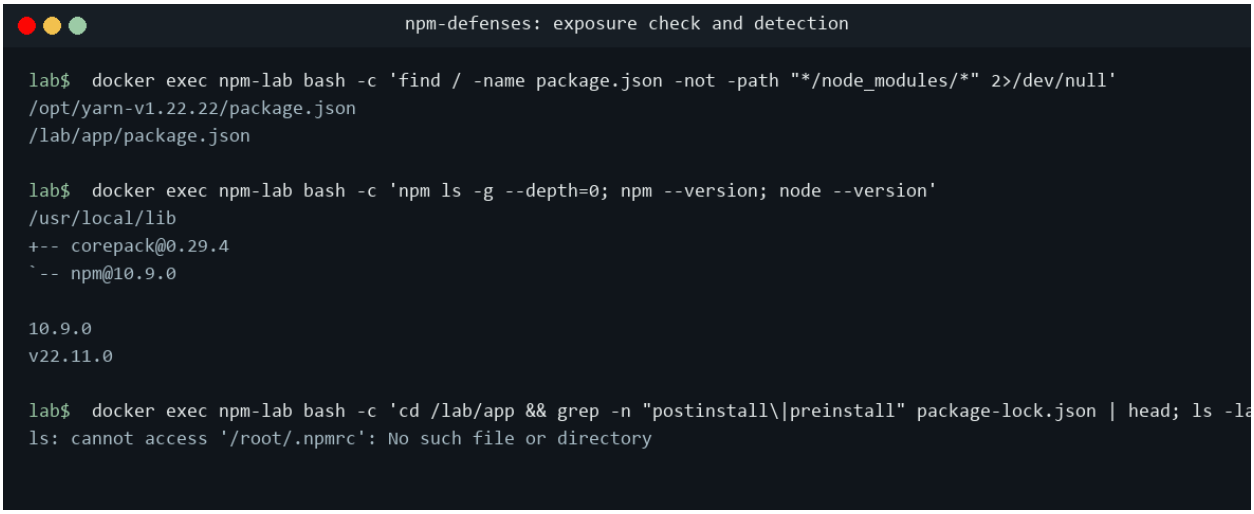
Lab captures

A lab run recorded part of Steps 1 to 4 on a lab host inside a throwaway `node:22.11.0-bookworm-slim` container (npm 10.9.0, Node v22.11.0), on 4 November 2026 UTC [175]. The sample project pinned two old direct dependencies, lodash 4.17.15 and minimist 1.2.5, and installed them with `--ignore-scripts`; nothing from the tree was executed. The prompt is a neutral `lab$`, and each command runs through `docker exec` into the container.

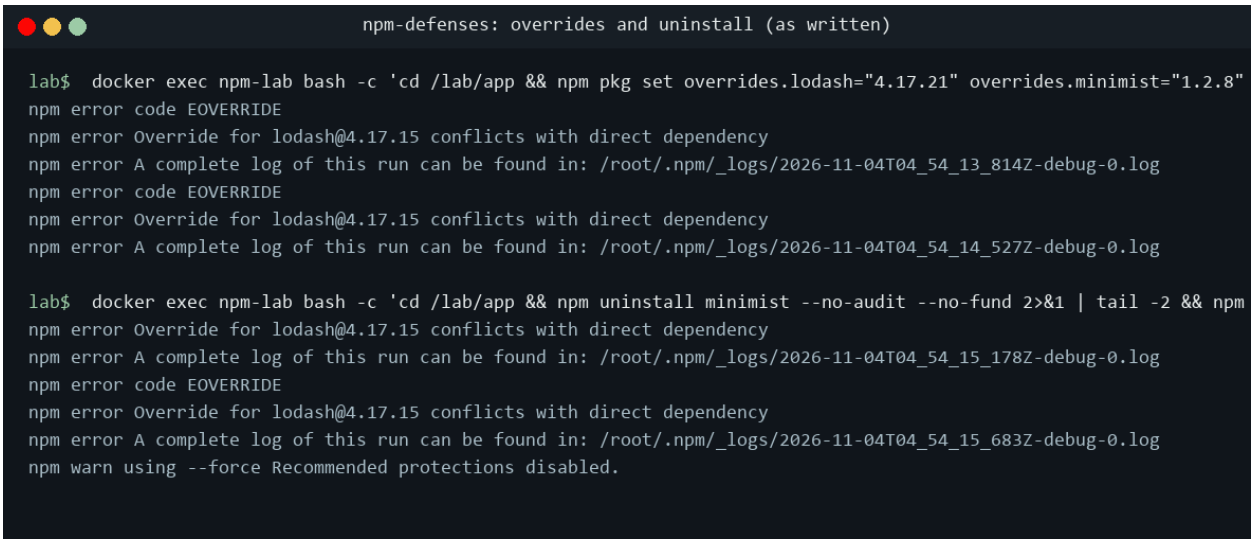


The first capture covers Steps 1 to 3. The JSON that `npm query "#lodash"` printed is cut from the image because it carries the package authors' contact details; the rest is the output as captured. Two readings matter for a defender. `npm audit` flagged lodash up to and including 4.17.23 and proposed 4.18.1, so 4.17.21, long the usual "safe" lodash target, is no longer one; take the fixed version from the advisory, not from memory (**Confirmed**, lab [175]). `npm audit signatures` verified registry signatures on both packages, which says the

tarballs are the ones the registry signed, not that they are free of known flaws: the same two packages failed `npm audit` (**Confirmed**, lab [175]). osv-scanner was not run, because the lab plan fetched an unpinned prebuilt binary; to use it, download a pinned release and check it against the release's published checksums before running it (**Inferred**).



The second capture is the exposure check: the install roots, the global packages and the npm and Node versions, no `preinstall` or `postinstall` hook recorded in the lockfile, and no `~/.npmrc` that could hold a publish token (**Confirmed**, lab [175]). On a real host the same commands run without the `docker exec` prefix.



The third capture is the lab plan's remediation step, and it failed. Overrides set on the two direct dependencies stopped `npm install` with `EOVERRIDE`, and because the bad override was now in `package.json`, the next `npm uninstall` failed the same way (**Confirmed**, lab [175]), as the npm documentation says it will [116]. Overrides are for transitive dependencies. For a direct dependency, change the dependency itself, for example `npm install lodash@4.18.1 minimist@1.2.8 --ignore-scripts`, or point the override at the direct spec with the `$` reference form [116] (**Inferred**, not run in the lab). The lab run did not record a successful remove, reinstall or verify step, so Step 4 has no screenshot of a clean result.

Part 8: MITRE mappings

ATT&CK models npm worms directly. S9008 Shai-Hulud (malware, v1.0, created 9 April 2026) is a supply-chain worm first reported in September 2025 that spreads through npm packages and GitHub, steals credentials and tokens and exfiltrates through GitHub Actions workflows; it maps 33 techniques and attributes no group (**Confirmed** [160]). S9043 Mini Shai-Hulud (v1.0, created 29 July 2026) is derived from S9008, used by G1056 TeamPCP since at least 2026, uses stolen npm and GitHub OIDC tokens to infect the victim's other packages and includes a targeted wiper; it maps 55 techniques (**Confirmed** [161]). The table below lists the techniques that tie to npm behavior in this study's record; the linked software pages carry the full lists. Every id was checked against the live ATT&CK page in the Source column or against the library's ATT&CK v20.0 data, with the tactic as the library's `technique_profiles.jsonl` records it, and each has a generated page [166].

ATT&CK Enterprise techniques

ATT&CK id	Name	Tactic (v20)	Dated event or report	Basis	Source
T1195.001	Compromise Software Dependencies and Development Tools	Initial Access	Shai-Hulud (2025-09-15); axios (2026-03-31); Mini Shai-Hulud (2026-04-29)	ATT&CK procedures: S9008, S9043, S9042 CanisterWorm, S9034 Tsundere Botnet, S1246 BeaverTail, G1056, G0082 (Axios and Mastra)	[162] [22] [35] [38]
T1204.005	Malicious Library	Execution	Contagious Interview packages (report 2025-10-10)	ATT&CK description names NPM; sole procedure G1052	[163] [24]
T1059.007	Command and Scripting Interpreter: JavaScript	Execution	Every payload in Part 2	ATT&CK software mappings S9008, S9043, S9042, S9041	[160] [161] [166]
T1546.016	Event Triggered Execution: Installer Packages	Persistence, Privilege Escalation	coa preinstall (2021-11-04); Shai-Hulud 2.0 preinstall (2025-11-24)	ATT&CK procedures: S9008 postinstall and preinstall hooks, G0082, G1056, S9041; the technique's description itself does not mention npm	[164] [7] [26]
T1677	Poisoned Pipeline Execution	Execution	`shai-hulud-workflow.yml` (2025-09-15); Nx workflow injection (2025-08-26)	ATT&CK procedures: S9008 (`.github/workflows/discussion.yaml`), S9043, S9042, G0082	[160] [22] [18]
T1552.001	Unsecured Credentials: Credentials In Files	Credential Access	TruffleHog and `process.env` dump (2025-09-15)	ATT&CK: S9008, S9043, S9041	[160] [22]
T1552.004	Unsecured Credentials: Private Keys	Credential Access	Mini Shai-Hulud (2026-04-29)	ATT&CK: S9043, S9042, S9041	[166] [38]
T1552.005	Unsecured Credentials: Cloud Instance Metadata API	Credential Access	Shai-Hulud waves (2025)	ATT&CK: S9008, S9043	[166]
T1552.007	Unsecured Credentials: Container API	Credential Access	Mini Shai-Hulud (2026)	ATT&CK: S9043, S9041	[166]
T1528	Steal Application Access Token	Credential Access	Nx `NPM_TOKEN` (2025-08-26); TanStack OIDC token from runner memory (2026-05-11)	ATT&CK: S9008, S9043	[160] [18] [43]
T1555.006	Credentials from Password Stores: Cloud Secrets Management Stores	Credential Access	Mini Shai-Hulud cloud and CI credentials (2026-04-29)	ATT&CK: S9008, S9043	[166] [38]

ATT&CK id	Name	Tactic (v20)	Dated event or report	Basis	Source
T1078.004	Valid Accounts: Cloud Accounts	Initial Access (also Persistence, Privilege Escalation, Stealth)	Shai-Hulud waves (2025)	ATT&CK: S9008, S9043	[160] [166]
T1550.001	Use Alternate Authentication Material: Application Access Token	Lateral Movement	Stolen tokens republish the victim's other packages (2025-09; 2026-04)	ATT&CK: S9008, S9043, S9042	[161] [22]
T1213.003	Data from Information Repositories: Code Repositories	Collection	Shai-Hulud waves	ATT&CK: S9008, S9043, S9041	[166]
T1567.001	Exfiltration Over Web Service: Exfiltration to Code Repository	Exfiltration	Shai-Hulud repository (2025-09-15); "Sha1-Hulud: The Second Coming" (2025-11); "Shai-Hulud: Here We Go Again" (2026-08-04)	ATT&CK procedures: S9008 repository named Shai-Hulud; S9043 new public repository in the victim account	[165] [22] [27] [47]
T1567.004	Exfiltration Over Web Service: Exfiltration Over Webhook	Exfiltration	webhook.site (2025-09-15)	ATT&CK: S9008	[160] [22] [23]
T1041	Exfiltration Over C2 Channel	Exfiltration	Shai-Hulud waves	ATT&CK: S9008, S9043, S9041	[166]
T1608.001	Stage Capabilities: Upload Malware	Resource Development	Malicious versions published to the registry	ATT&CK: S9008	[166]
T1036.005	Masquerading: Match Legitimate Resource Name or Location	Stealth	Shai-Hulud waves	ATT&CK: S9008, S9043, S9042, S9041	[166]
T1027	Obfuscated Files or Information	Stealth	Whitespace padding in `@react-native-aria` (2025-06-06); obfuscated node-ipc stealer (2026-05-14)	ATT&CK: S9008; the two events are Inferred fits	[166] [16] [44]
T1059.004	Command and Scripting Interpreter: Unix Shell	Execution	First-wave bash script, likely LLM-generated (2025-09)	ATT&CK: S9008, S9042, S9041	[166] [27]
T1485	Data Destruction	Impact	Shai-Hulud 2.0 home-directory destruction (2025-11)	ATT&CK: S9008, S9043 (targeted wiper), S9042	[161] [27]
T1102.001	Web Service: Dead Drop Resolver	Command and Control	keyv wave C2 resolved from an Ethereum contract (2026-08-04)	ATT&CK: S9043, S9042; the keyv event is an Inferred fit	[166] [47]
T1657	Financial Theft	Impact	event-stream wallet keys (2018); chalk/debug address swapping (2025-09-08)	ATT&CK: S1246 BeaverTail, S9041; the two events are Inferred fits	[166] [1] [21]

ATT&CK id	Name	Tactic (v20)	Dated event or report	Basis	Source
T1195.002	Compromise Software Supply Chain	Initial Access	Nx publish workflow (2025-08-26); TanStack release pipeline (2026-05-11)	Inferred: the live page has no npm procedure	[18] [43]
T1566	Phishing	Initial Access	eslint-config-prettier (2025-07-18); chalk/debug (2025-09-08)	Inferred	[17] [20]
T1583.001	Acquire Infrastructure: Domains	Resource Development	`npmjs.help` registered 2025-09-05	Inferred	[21]
T1078	Valid Accounts	Initial Access (also Persistence, Privilege Escalation, Stealth)	ua-parser-js, coa, rc (2021); lottie-player (2024-10-30); axios (2026-03-31)	Inferred	[5] [58] [14] [35]
T1585	Establish Accounts	Resource Development	180+ fake personas (2025-10-10); Jade Sleet personas (2023-07-18)	Inferred	[24] [12]
T1565.002	Data Manipulation: Transmitted Data Manipulation	Impact	Swapped transaction recipients (2025-09-08)	Inferred	[21]

ATT&CK for ICS and Mobile do not apply. The record supports the sub-techniques T1078.004 and T1204.005 specifically; T1078 is listed separately as the study's inference for hijacked maintainer accounts, and the parent T1204 is context only.

Group, software and campaign objects

Object	Id	Relation to this study	Source
Software	S9008 Shai-Hulud	The September 2025 worm; 33 techniques; no group attributed	[160]
Software	S9043 Mini Shai-Hulud	Derived from S9008; used by TeamPCP; 55 techniques	[161]
Software	S9042 CanisterWorm	Spreads by publishing npm packages (T1195.001)	[162] [166]
Software	S9041 TeamPCP Cloud Stealer	Injects install scripts into `package.json` (T1546.016)	[164]
Software	S9034 Tsundere Botnet	Used npm to download malicious packages (T1195.001)	[162]
Software	S1246 BeaverTail	Distributed through npm packages; rebuilt in memory by Contagious Interview loaders	[162] [24]
Software	S1245 InvisibleFerret	Typically fetched by BeaverTail in the Contagious Interview packages	[24] [166]
Group	G1056 TeamPCP	Uses S9043; ATT&CK procedures for T1195.001, T1546.016 and T1677	[161] [164] [166]
Group	G0082 APT38	ATT&CK says it injected malicious dependencies, with post-install hooks, into the Axios and Mastra npm packages. The axios maintainer's post-mortem names no actor, though in a later comment on the same issue the maintainer likened the approach to Google's write-up of UNC1069, and Socket saw no link to TeamPCP	[162] [164] [35] [36]
Group	G1052 Contagious Interview	The only procedure example on T1204.005	[163]
Campaigns	none	No campaign in the library's ATT&CK data is about npm; its only supply-chain campaigns are C0024 SolarWinds and C0057 3CX	[166]

CWE and CAPEC

Id	Name	Where it appears	Basis	Source
CWE-506	Embedded Malicious Code	coa (GHSA-73qr-pfmq-6rp8); node-ipc 9.1.6 (GHSA-pvh2-rg5g-69v7)	GHSA weakness field (Confirmed)	[6] [45]
CWE-494	Download of Code Without Integrity Check	Listed by CAPEC-538 and CAPEC-695	CAPEC (Confirmed)	[167] [168]
CWE-829	Inclusion of Functionality from Untrusted Control Sphere	Listed by CAPEC-538 and CAPEC-695	CAPEC (Confirmed)	[167] [168]
CVE-2022-23812	node-ipc 10.1.1 and 10.1.2	The CVE id assigned to the 2022 protestware	GHSA (Confirmed)	[10]
CAPEC-538	Open-Source Library Manipulation	Malicious code hidden in open-source libraries; ChildOf CAPEC-444; maps to T1195.001 (CAPEC 3.9)	Mapping Confirmed; fit to every hijacked package Inferred	[167]
CAPEC-695	Repo Jacking	Re-registered repository paths that hardcoded dependencies pull from; maps to T1195.001; relevant to git-URL dependencies	Mapping Confirmed; fit Inferred	[168]
CAPEC-630	TypoSquatting	Defined for domain names, not package names; no CWE and no direct ATT&CK mapping. Package look-alikes fit CAPEC-538 or CAPEC-691 better	CAPEC (Confirmed); remapping Inferred	[169]

The library has no per-id CWE pages; see the CWE Reference.

D3FEND countermeasures

The library's D3FEND data maps T1195.001 to four countermeasures: Software Update, Asset Vulnerability Enumeration, Software Inventory and Restore Software. It maps T1552 to Credential Hardening, Credential Rotation, Credential Revocation, Multi-factor Authentication and related techniques; T1528 adds Token Binding and Token-based Authentication; T1552.001 includes File Analysis and File Integrity Monitoring; and T1567.001 maps to Outbound Traffic Filtering and network analyses (**Confirmed** [166]). The live D3FEND API confirms D3-SU, D3-FA, D3-CH, D3-CRO, D3-TL and D3-SWI [170]. The library's data has no D3FEND rows for T1059.007, T1204.005, T1546.016 or T1677 [166]. Pairing Trusted Library (D3-TL) with an internal vetted registry or proxy is the study's own mapping: D3FEND links D3-TL only to T1505.001 (**Inferred** [171]). D3FEND has no "software update verification" countermeasure; signature and provenance checks are described in this study's own words and SLSA's [144].

OWASP CI/CD risks

The OWASP Top 10 CI/CD Security Risks run from CICD-SEC-1 Insufficient Flow Control Mechanisms to CICD-SEC-10 Insufficient Logging and Visibility (**Confirmed** [172]). CICD-SEC-3 Dependency Chain Abuse names dependency confusion, dependency hijacking, typosquatting and brandjacking and cites ua-parser-js, coa and rc as hijacked npm libraries [173]. CICD-SEC-6 Insufficient Credential Hygiene recommends least privilege per credential, no reuse, temporary credentials, restricting where credentials can be used, secret scanning at the IDE, on push and across history, and masking secrets in logs; it does not mention OIDC [174]. The Shai-Hulud worms touch CICD-SEC-3 (hijacked packages), CICD-SEC-4 Poisoned Pipeline Execution (the same name ATT&CK uses for T1677), CICD-SEC-6 (token theft) and CICD-SEC-9 Improper Artifact Integrity Validation (**Inferred** [172] [22] [26]).

Navigator layer

navigator/library-layers/npm-supply-chain-attacks.json carries the 30 technique entries above (ATT&CK v20, domain `enterprise-attack`). Score 100 marks an ATT&CK-published link to an npm worm, loader or group in the record; score 50 marks the study's labelled inferences. Each entry's comment names the source numbers.

MITRE ATT&CK, CWE, CAPEC and D3FEND content is used under the terms recorded in THIRD_PARTY_NOTICES.md.

Cross-cutting themes

- **The trust unit is a person with a login.** From the event-stream handover to the axios RAT, the attacker's goal was a maintainer's publishing rights. Every registry control that worked raised the cost of using those rights: 2FA, session tokens, trusted publishing, staged approval [58] [73] [122] [77] (**Inferred**).
- **Install time is where the code ran, and it is now closing.** npm 12, pnpm 10 and later and Yarn Berry all block dependency build scripts by default [79] [89] [130]. The attacks that are left run on `require()`, in the browser or through editors and agents [44] [21] [19] [48] (**Inferred**).
- **Speed cuts both ways.** Worms published hundreds of versions in hours [22] [47], but the registry and the research community found most bad versions in minutes to hours [38] [43] [90]. A short wait before adopting a new version beat most of the record (**Inferred**).
- **Provenance tells you where a package was built, not whether the build was honest.** It caught Nx and eslint-config-prettier; it said nothing about TanStack or keyv [18] [17] [43] [47].
- **CI is part of the supply chain.** `pull_request_target`, shared caches, runner memory and self-hosted runners appear in the Nx, PostHog, TanStack, Ultralytics and Shai-Hulud 2.0 cases [18] [30] [43] [55] [26].
- **The developer machine is the soft target.** The 2026 cases moved to the maintainer's own computer (a RAT, a VS Code extension) and to the tools on it (AI CLIs, editor tasks, agent hooks) [35] [46] [18] [48].
- **What is overstated, and what is understated.** Typosquatting gets attention, but the largest waves here were legitimate, popular packages published with stolen rights, not look-alikes. The tea.xyz campaign's 150,000 packages are a spam and token-farming problem, and AWS says they contain no overtly malicious code [25]. Understated: state-linked delivery through fake packages and fake recruiters, which GitHub, Socket and ATT&CK all document [12] [24] [162], and the risk from tokens that bypass 2FA, which drove the August 2026 worm [48] (**Inferred** in the weighting; the facts are Confirmed).

Sources

Sources 1 to 174 were opened on 3 November 2026; source 175 is the lab capture.

#	Source	URL	Opened	Grade
1	npm blog, "Details about the event-stream incident" (2018-11-27)	https://blog.npmjs.org/post/180565383195/details-about-the-event-stream-incident	2026-11-03	Confirmed
2	GitHub Advisory GHSA-mh6f-8j2x-4483 (event-stream, flatmap-stream)	https://github.com/advisories/GHSA-mh6f-8j2x-4483	2026-11-03	Confirmed
3	Alex Birsan, "Dependency Confusion" (2021-02-09)	https://medium.com/@alex.birsan/dependency-confusion-4a5d60fec610	2026-11-03	Demonstrated (incident); Confirmed (mechanism)
4	CISA alert, malware in ua-parser-js (2021-10-22)	https://www.cisa.gov/news-events/alerts/2021/10/22/malware-discovered-popular-npm-package-ua-parser-js	2026-11-03	Confirmed
5	ua-parser-js issue #536 (maintainer statement)	https://github.com/faisalman/ua-parser-js/issues/536	2026-11-03	Confirmed
6	GitHub Advisory GHSA-73qr-pfmq-6rp8 (coa)	https://github.com/advisories/GHSA-73qr-pfmq-6rp8	2026-11-03	Confirmed
7	coa issue #99	https://github.com/veged/coa/issues/99	2026-11-03	Confirmed
8	GitHub Advisory GHSA-g2q5-5433-rhrf (rc)	https://github.com/advisories/GHSA-g2q5-5433-rhrf	2026-11-03	Confirmed
9	Snyk, colors and faker (2022-01-09)	https://snyk.io/blog/open-source-npm-packages-colors-faker/	2026-11-03	Confirmed
10	GitHub Advisory GHSA-97m3-w2cp-4xx6 (node-ipc, CVE-2022-23812)	https://github.com/advisories/GHSA-97m3-w2cp-4xx6	2026-11-03	Confirmed
11	Snyk, peacenotwar and node-ipc (2022-03-16)	https://snyk.io/blog/peacenotwar-malicious-npm-node-ipc-package-vulnerability/	2026-11-03	Confirmed
12	GitHub blog, social engineering campaign alert (2023-07-18)	https://github.blog/security/vulnerability-research/security-alert-social-engineering-campaign-targets-technology-industry-employees/	2026-11-03	Confirmed
13	lottie-player issue #254	https://github.com/LottieFiles/lottie-player/issues/254	2026-11-03	Confirmed
14	The Hacker News, LottieFiles warning (October 2024)	https://thehackernews.com/2024/10/lottiefiles-issues-warning-about.html	2026-11-03	Reported
15	Rspack issue #8767 (team notice)	https://github.com/web-infra-dev/rspack/issues/8767	2026-11-03	Confirmed
16	Aikido, React Native Aria supply-chain attack (2025-06-07)	https://www.aikido.dev/blog/supply-chain-attack-on-react-native-aria-ecosystem	2026-11-03	Confirmed

#	Source	URL	Opened	Grade
17	eslint-config-prettier issue #339	https://github.com/prettier/eslint-config-prettier/issues/339	2026-11-03	Confirmed
18	Nx, s1ngularity post-mortem (2025-09-05)	https://nx.dev/blog/s1ngularity-postmortem	2026-11-03	Confirmed
19	Nx security advisory GHSA-cxm3-wv7p-598c	https://github.com/nrwl/nx/security/advisories/GHSA-cxm3-wv7p-598c	2026-11-03	Confirmed
20	debug issue #1005 (maintainer timeline)	https://github.com/debug-js/debug/issues/1005	2026-11-03	Confirmed
21	Aikido, debug and chalk packages compromised (2025-09-08)	https://www.aikido.dev/blog/npm-debug-and-chalk-packages-compromised	2026-11-03	Confirmed
22	StepSecurity, @ctrl/tinycolor and 40+ packages compromised (2025-09-15)	https://www.stepsecurity.io/blog/ctrl-tinycolor-and-40-npm-packages-compromised	2026-11-03	Confirmed
23	CISA alert, widespread supply-chain compromise impacting npm (2025-09-23)	https://www.cisa.gov/news-events/alerts/2025/09/23/widespread-supply-chain-compromise-impacting-npm-ecosystem	2026-11-03	Confirmed
24	Socket, Contagious Interview, 338 malicious npm packages (2025-10-10)	`socket.dev/blog/north-korea-contagious-interview-campaign-338-malicious-npm-packages` (not linked: the host returns 403 to automated link checks)	2026-11-03	Confirmed
25	AWS Security Blog, Amazon Inspector and the tea.xyz token-farming campaign (2025-11-13)	https://aws.amazon.com/blogs/security/amazon-inspector-detects-over-150000-malicious-packages-linked-to-token-farming-campaign/	2026-11-03	Confirmed
26	Wiz, Shai-Hulud 2.0	https://www.wiz.io/blog/shai-hulud-2-0-ongoing-supply-chain-attack	2026-11-03	Confirmed
27	Unit 42, npm supply-chain attack	https://unit42.paloaltonetworks.com/npm-supply-chain-attack/	2026-11-03	Confirmed
28	Check Point, Shai-Hulud 2.0 (2025-11-25)	https://blog.checkpoint.com/research/shai-hulud-2-0-inside-the-second-coming-the-most-aggressive-npm-supply-chain-attack-of-2025/	2026-11-03	Confirmed
29	CERT-FR, CERTFR-2025-ACT-051 (2025-11-27)	https://cert.cyber.gouv.fr/actualite/CERTFR-2025-ACT-051/	2026-11-03	Confirmed
30	PostHog, Shai-Hulud post-mortem	https://posthog.com/blog/nov-24-shai-hulud-attack-post-mortem	2026-11-03	Confirmed
31	Socket, Shai-Hulud strikes again (Maven Central update, 2025-11-25)	`socket.dev/blog/shai-hulud-strikes-again-v2` (not linked: the host returns 403 to automated link checks)	2026-11-03	Reported
32	Wiz IOC list, shai-hulud-2-packages.csv	https://github.com/wiz-sec-public/wiz-research-iocs/blob/main/reports/shai-hulud-2-packages.csv	2026-11-03	Confirmed

#	Source	URL	Opened	Grade
33	GitHub Advisory GHSA-fw8c-xr5c-95f9 (axios)	https://github.com/advisories/GHSA-fw8c-xr5c-95f9	2026-11-03	Confirmed
34	GitHub Advisory GHSA-2x9r-6wxq-hrr7 (plain-crypto-js)	https://github.com/advisories/GHSA-2x9r-6wxq-hrr7	2026-11-03	Confirmed
35	axios issue #10636, post-mortem	https://github.com/axios/axios/issues/10636	2026-11-03	Confirmed
36	Socket, axios npm package compromised (2026-03-31)	`socket.dev/blog/axios-npm-package-compromised` (not linked: the host returns 403 to automated link checks)	2026-11-03	Confirmed
37	Datadog indicators of compromise, axios	https://github.com/DataDog/indicators-of-compromise/tree/main/axios-npm-supply-chain-compromise	2026-11-03	Confirmed
38	Onapsis, SAP CAP Mini Shai-Hulud (2026-04-29)	https://onapsis.com/blog/sap-cap-mini-shai-hulud-supply-chain-attack/	2026-11-03	Confirmed
39	GitHub Advisory GHSA-2h7r-x9v2-q52f (@cap-js/sqlite)	https://github.com/advisories/GHSA-2h7r-x9v2-q52f	2026-11-03	Confirmed
40	GitHub Advisory GHSA-72vc-j35j-p28h (@cap-js/db-service)	https://github.com/advisories/GHSA-72vc-j35j-p28h	2026-11-03	Confirmed
41	GitHub Advisory GHSA-4594-wxqv-j3pm (intercom-client)	https://github.com/advisories/GHSA-4594-wxqv-j3pm	2026-11-03	Confirmed
42	Kudelski Security, Mini Shai-Hulud	https://kudelskisecurity.com/research/mini-shai-hulud-supply-chain-attack	2026-11-03	Reported
43	TanStack, npm supply-chain compromise post-mortem	https://tanstack.com/blog/npm-supply-chain-compromise-postmortem	2026-11-03	Confirmed
44	StepSecurity, node-ipc supply-chain attack (2026)	https://www.stepsecurity.io/blog/node-ipc-npm-supply-chain-attack	2026-11-03	Confirmed
45	GitHub Advisory GHSA-pvh2-rg5g-69v7 (node-ipc 9.1.6)	https://github.com/advisories/GHSA-pvh2-rg5g-69v7	2026-11-03	Confirmed
46	Red Hat security bulletin RHSB-2026-006	https://access.redhat.com/security/vulnerabilities/RHSB-2026-006	2026-11-03	Confirmed
47	Aikido, keyv and friends compromised (2026-08-04)	https://www.aikido.dev/blog/keyv-and-friends-compromised-in-npm-supply-chain-attack	2026-11-03	Confirmed
48	Elastic Security Labs, Shai-Hulud CHAINDROP (2026-08-06)	https://www.elastic.co/security-labs/shai-hulud-chaindrop-npm-supply-chain	2026-11-03	Confirmed
49	GitHub Advisory GHSA-3p9h-f68w-m6fx (keyv)	https://github.com/advisories/GHSA-3p9h-f68w-m6fx	2026-11-03	Confirmed
50	GitHub Advisory GHSA-8jxr-wprf-45g8 (flat-cache)	https://github.com/advisories/GHSA-8jxr-wprf-45g8	2026-11-03	Confirmed
51	GitHub Advisory GHSA-p7m5-96hg-gppj (cache-manager)	https://github.com/advisories/GHSA-p7m5-96hg-gppj	2026-11-03	Confirmed

#	Source	URL	Opened	Grade
52	Wiz IOC list, keyv-packages.csv	https://github.com/wiz-sec-public/wiz-research-iocs/blob/main/reports/keyv-packages.csv	2026-11-03	Confirmed
53	GitHub Advisory GHSA-9333-3c4x-x3h5 (@subql/common)	https://github.com/advisories/GHSA-9333-3c4x-x3h5	2026-11-03	Confirmed
54	GitHub Advisory GHSA-rqxj-g25x-4v9v (tensorlake)	https://github.com/advisories/GHSA-rqxj-g25x-4v9v	2026-11-03	Confirmed
55	PyPI blog, Ultralytics attack analysis (2024-12-11)	https://blog.pypi.org/posts/2024-12-11-ultralytics-attack-analysis/	2026-11-03	Confirmed
56	GitHub Changelog, Dependabot automated security fixes (2019-05-23)	https://github.blog/changelog/2019-05-23-dependabot-automated-security-fixes/	2026-11-03	Confirmed
57	GitHub Changelog, npm advisory database joins the GitHub Advisory Database (2021-10-03)	https://github.blog/changelog/2021-10-03-the-npm-advisory-database-is-now-part-of-the-github-advisory-database/	2026-11-03	Confirmed
58	GitHub blog, "GitHub's commitment to npm ecosystem security" (2021-11-15)	https://github.blog/open-source/githubs-commitment-to-npm-ecosystem-security/	2026-11-03	Confirmed
59	GitHub blog, enhanced login verification and 2FA enforcement (2021-12-07)	https://github.blog/open-source/enrolling-npm-publishers-enhanced-login-verification-two-factor-authentication-enforcement/	2026-11-03	Confirmed
60	GitHub blog, top-100 npm maintainers require 2FA (2022-02-01)	https://github.blog/security/supply-chain-security/top-100-npm-package-maintainers-require-2fa-additional-security/	2026-11-03	Confirmed
61	GitHub Changelog, all npm accounts enrolled in login verification (2022-03-01)	https://github.blog/changelog/2022-03-01-all-npm-accounts-are-now-enrolled-in-login-verification/	2026-11-03	Confirmed
62	GitHub Changelog, May 2022 (top-500 maintainers require 2FA, 2022-05-31)	https://github.blog/changelog/month/05-2022/	2026-11-03	Confirmed
63	GitHub Changelog, malware advisories (2022-06-15)	https://github.blog/changelog/2022-06-15-advisory-database-supports-malware-advisories/	2026-11-03	Confirmed
64	GitHub blog, even more security enhancements to npm (2022-07-26)	https://github.blog/news-insights/product-news/introducing-even-more-security-enhancements-to-npm/	2026-11-03	Confirmed
65	GitHub Changelog, high-impact maintainers require 2FA (2022-11-01)	https://github.blog/changelog/2022-11-01-high-impact-package-maintainers-now-require-2fa/	2026-11-03	Confirmed
66	GitHub Changelog, granular access tokens GA (2023-03-21)	https://github.blog/changelog/2023-03-21-general-availability-of-granular-access-token-on-npm/	2026-11-03	Confirmed
67	GitHub blog, introducing npm package provenance (2023-04-19)	https://github.blog/security/supply-chain-security/introducing-npm-package-provenance/	2026-11-03	Confirmed

#	Source	URL	Opened	Grade
68	GitHub Changelog, npm provenance GA (2023-09-26)	https://github.blog/changelog/2023-09-26-npm-provenance-general-availability/	2026-11-03	Confirmed
69	GitHub Changelog, npm trusted publishing with OIDC GA (2025-07-31)	https://github.blog/changelog/2025-07-31-npm-trusted-publishing-with-oidc-is-generally-available/	2026-11-03	Confirmed
70	GitHub blog, "Our plan for a more secure npm supply chain" (2025-09-22)	https://github.blog/security/supply-chain-security/our-plan-for-a-more-secure-npm-supply-chain/	2026-11-03	Confirmed
71	GitHub Changelog, npm authentication and token changes (2025-09-29)	https://github.blog/changelog/2025-09-29-strengthening-npm-security-important-changes-to-authentication-and-token-management/	2026-11-03	Confirmed
72	GitHub Changelog, classic token creation disabled (2025-11-05)	https://github.blog/changelog/2025-11-05-npm-security-update-classic-token-creation-disabled-and-granular-token-changes/	2026-11-03	Confirmed
73	GitHub Changelog, classic tokens revoked, session-based auth (2025-12-09)	https://github.blog/changelog/2025-12-09-npm-classic-tokens-revoked-session-based-auth-and-cli-token-management-now-available/	2026-11-03	Confirmed
74	npm CLI release v11.10.0 (with v11.17.0)	https://github.com/npm/cli/releases/tag/v11.10.0	2026-11-03	Confirmed
75	GitHub Changelog, bulk trusted publishing and script security (2026-02-18)	https://github.blog/changelog/2026-02-18-npm-bulk-trusted-publishing-config-and-script-security-now-generally-available/	2026-11-03	Confirmed
76	GitHub Changelog, Dependabot detects malware in npm dependencies (2026-03-17)	https://github.blog/changelog/2026-03-17-dependabot-now-detects-malware-in-npm-dependencies/	2026-11-03	Confirmed
77	GitHub Changelog, staged publishing and install-time controls (2026-05-22)	https://github.blog/changelog/2026-05-22-staged-publishing-and-new-install-time-controls-for-npm/	2026-11-03	Confirmed
78	GitHub Changelog, upcoming breaking changes for npm v12 (2026-06-09)	https://github.blog/changelog/2026-06-09-upcoming-breaking-changes-for-npm-v12/	2026-11-03	Confirmed
79	npm CLI release v12.0.0 (2026-07-08)	https://github.com/npm/cli/releases/tag/v12.0.0	2026-11-03	Confirmed
80	npm CLI CHANGELOG.md	https://raw.githubusercontent.com/npm/cli/latest/CHANGELOG.md	2026-11-03	Confirmed
81	GitHub Changelog, npm install-time security and bypass-2FA deprecation (2026-07-08)	https://github.blog/changelog/2026-07-08-npm-install-time-security-and-gat-bypass2fa-deprecation/	2026-11-03	Confirmed
82	GitHub Changelog, publish-time malware scanning and dual-use metadata (2026-07-28)	https://github.blog/changelog/2026-07-28-npm-publish-time-malware-scanning-and-dual-use-metadata/	2026-11-03	Confirmed

#	Source	URL	Opened	Grade
83	GitHub Changelog, Dependabot alerts on malicious packages across more ecosystems (2026-07-28)	https://github.blog/changelog/2026-07-28-dependabot-alerts-on-malicious-packages-across-more-ecosystems/	2026-11-03	Confirmed
84	GitHub Changelog, restricting bypass-2FA granular tokens (2026-07-31)	https://github.blog/changelog/2026-07-31-restricting-npm-bypass-2fa-granular-access-tokens/	2026-11-03	Confirmed
85	GitHub Changelog, multiple trusted publishing configurations (2026-09-03)	https://github.blog/changelog/2026-09-03-multiple-trusted-publishing-configurations-for-npm/	2026-11-03	Confirmed
86	GitHub Changelog, stage-only npm tokens (2026-09-18)	https://github.blog/changelog/2026-09-18-stage-only-npm-tokens-for-safer-automation/	2026-11-03	Confirmed
87	GitHub Changelog, unvalidated trusted publishing configurations expire (2026-10-02)	https://github.blog/changelog/2026-10-02-unvalidated-npm-trusted-publishing-configurations-now-expire/	2026-11-03	Confirmed
88	GitHub Changelog, staged publishing creates new packages (2026-10-02)	https://github.blog/changelog/2026-10-02-npm-staged-publishing-now-supports-creating-new-packages/	2026-11-03	Confirmed
89	pnpm release v10.0.0 (2025-01-07)	https://github.com/pnpm/pnpm/releases/tag/v10.0.0	2026-11-03	Confirmed
90	pnpm release v10.16.0 (2025-09-12)	https://github.com/pnpm/pnpm/releases/tag/v10.16.0	2026-11-03	Confirmed
91	pnpm release v10.21.0 (2025-11-09)	https://github.com/pnpm/pnpm/releases/tag/v10.21.0	2026-11-03	Confirmed
92	pnpm release v10.26.0 (2025-12-15)	https://github.com/pnpm/pnpm/releases/tag/v10.26.0	2026-11-03	Confirmed
93	pnpm release v11.0.0 (2026-04-28)	https://github.com/pnpm/pnpm/releases/tag/v11.0.0	2026-11-03	Confirmed
94	Yarn release @yarnpkg/cli 4.9.0 (2025-04-09)	https://github.com/yarnpkg/berry/releases/tag/%40yarnpkg%2Fcli%2F4.9.0	2026-11-03	Confirmed
95	Yarn release @yarnpkg/cli 4.10.0 (2025-09-18)	https://github.com/yarnpkg/berry/releases/tag/%40yarnpkg%2Fcli%2F4.10.0	2026-11-03	Confirmed
96	Yarn release @yarnpkg/cli 4.16.0 (2026-06-02)	https://github.com/yarnpkg/berry/releases/tag/%40yarnpkg%2Fcli%2F4.16.0	2026-11-03	Confirmed
97	npm registry and GitHub release endpoints, current versions (npm shown)	https://registry.npmjs.org/npm/latest	2026-11-03	Confirmed
98	npm CLI config definitions (definitions.js)	https://github.com/npm/cli/blob/latest/workspaces/config/lib/definitions/definitions.js	2026-11-03	Confirmed
99	npm docs source, npm-dist-tag	https://github.com/npm/cli/blob/latest/docs/lib/content/commands/npm-dist-tag.md	2026-11-03	Confirmed
100	npm docs source, scope	https://github.com/npm/cli/blob/latest/docs/lib/content/using-npm/scope.md	2026-11-03	Confirmed

#	Source	URL	Opened	Grade
101	npm docs source, package.json	https://github.com/npm/cli/blob/latest/docs/lib/content/configuring-npm/package-json.md	2026-11-03	Confirmed
102	npm registry manifests (esbuild, @esbuild/, *@swc/core, sharp, @img/sharp-*, @rollup/rollup-linux-arm64-gnu)	https://registry.npmjs.org/esbuild/latest	2026-11-03	Confirmed
103	esbuild, Getting Started	https://esbuild.github.io/getting-started/	2026-11-03	Confirmed
104	prebuild-install README	https://github.com/prebuild/prebuild-install	2026-11-03	Confirmed
105	npm docs v11, config	https://docs.npmjs.com/cli/v11/using-npm/config	2026-11-03	Confirmed
106	npm docs v12, config	https://docs.npmjs.com/cli/v12/using-npm/config	2026-11-03	Confirmed
107	npm docs v12, npm install-scripts	https://docs.npmjs.com/cli/v12/commands/npm-install-scripts	2026-11-03	Confirmed
108	npm docs v11, npm rebuild	https://docs.npmjs.com/cli/v11/commands/npm-rebuild	2026-11-03	Confirmed
109	npm docs v12, npm ls	https://docs.npmjs.com/cli/v12/commands/npm-ls	2026-11-03	Confirmed
110	npm docs v11, npm query	https://docs.npmjs.com/cli/v11/commands/npm-query	2026-11-03	Confirmed
111	npm docs v11, npm explain	https://docs.npmjs.com/cli/v11/commands/npm-explain	2026-11-03	Confirmed
112	npm docs v12, npm audit	https://docs.npmjs.com/cli/v12/commands/npm-audit	2026-11-03	Confirmed
113	npm docs v11, npm audit	https://docs.npmjs.com/cli/v11/commands/npm-audit	2026-11-03	Confirmed
114	npm docs, verifying registry signatures	https://docs.npmjs.com/verifying-registry-signatures	2026-11-03	Confirmed
115	npm docs v11, npm uninstall	https://docs.npmjs.com/cli/v11/commands/npm-uninstall	2026-11-03	Confirmed
116	npm docs v11, package.json (overrides)	https://docs.npmjs.com/cli/v11/configuring-npm/package-json	2026-11-03	Confirmed
117	npm docs v11, npm ci	https://docs.npmjs.com/cli/v11/commands/npm-ci	2026-11-03	Confirmed
118	npm docs v11, npm cache	https://docs.npmjs.com/cli/v11/commands/npm-cache	2026-11-03	Confirmed
119	npm docs v11, npm token	https://docs.npmjs.com/cli/v11/commands/npm-token	2026-11-03	Confirmed
120	npm docs v11, npm sbom	https://docs.npmjs.com/cli/v11/commands/npm-sbom	2026-11-03	Confirmed

#	Source	URL	Opened	Grade
121	npm docs, generating provenance statements	https://docs.npmjs.com/generating-provenance-statements	2026-11-03	Confirmed
122	npm docs, trusted publishers	https://docs.npmjs.com/trusted-publishers	2026-11-03	Confirmed
123	npm docs, configuring two-factor authentication	https://docs.npmjs.com/configuring-two-factor-authentication	2026-11-03	Confirmed
124	pnpm docs, pnpm audit	https://pnpm.io/cli/audit	2026-11-03	Confirmed
125	pnpm docs, pnpm list (with pnpm why, pnpm root)	https://pnpm.io/cli/list	2026-11-03	Confirmed
126	pnpm docs, dependency resolution settings	https://pnpm.io/settings/dependency-resolution	2026-11-03	Confirmed
127	pnpm docs, build settings	https://pnpm.io/settings/build	2026-11-03	Confirmed
128	Yarn classic docs, yarn audit (with why, list, cache, global and selective version resolutions)	https://classic.yarnpkg.com/en/docs/cli/audit	2026-11-03	Confirmed
129	Yarn docs, yarn npm audit (with why, install, cache clean and manifest)	https://yarnpkg.com/cli/npm/audit	2026-11-03	Confirmed
130	Yarn docs, .yarnrc.yml settings	https://yarnpkg.com/configuration/yarnrc	2026-11-03	Confirmed
131	Bun docs, bun audit	https://bun.com/docs/pm/cli/audit	2026-11-03	Confirmed
132	Anchore, grype CLI reference	https://oss.anchore.com/docs/reference/grype/cli/	2026-11-03	Confirmed
133	OWASP Dependency-Check project page	https://owasp.org/www-project-dependency-check/	2026-11-03	Confirmed
134	Dependency-Check repository (README and analyzer docs)	https://github.com/dependency-check/DependencyCheck	2026-11-03	Confirmed
135	OWASP CycloneDX project page (with the @cyclonedx/cyclonedx-npm registry entry)	https://owasp.org/www-project-cyclonedx/	2026-11-03	Confirmed
136	nvm README	https://github.com/nvm-sh/nvm	2026-11-03	Confirmed
137	GNU find(1), man7.org	https://man7.org/linux/man-pages/man1/find.1.html	2026-11-03	Confirmed
138	FreeBSD find(1)	https://man.freebsd.org/cgi/man.cgi?query=find&sektion=1	2026-11-03	Confirmed
139	OpenSSF package manager best practices, npm	https://github.com/ossf/package-manager-best-practices/blob/main/published/npm.md	2026-11-03	Confirmed
140	Verdaccio docs, package access	https://verdaccio.org/docs/packages/	2026-11-03	Confirmed
141	JFrog Curation, list of available conditions	https://docs.jfrog.com/security/docs/list-of-available-conditions	2026-11-03	Confirmed
142	Sonatype Firewall quarantine	https://help.sonatype.com/en/firewall-quarantine.html	2026-11-03	Confirmed

#	Source	URL	Opened	Grade
143	GitHub Docs, working with the npm registry (GitHub Packages)	https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-npm-registry	2026-11-03	Confirmed
144	SLSA v1.2, Build Track basics	https://slsa.dev/spec/v1.2/build-track-basics	2026-11-03	Confirmed
145	GitHub Docs, secure use reference for GitHub Actions	https://docs.github.com/en/actions/reference/security/secure-use	2026-11-03	Confirmed
146	GitHub Docs, push protection	https://docs.github.com/en/code-security/concepts/secret-security/push-protection	2026-11-03	Confirmed
147	GitHub Docs, export dependencies as an SBOM	https://docs.github.com/en/code-security/how-tos/secure-your-supply-chain/establish-provenance-and-integrity/export-dependencies-as-sbom	2026-11-03	Confirmed
148	GitHub Docs, malware alerts	https://docs.github.com/en/code-security/concepts/supply-chain-security/malware-alerts	2026-11-03	Confirmed
149	GitHub Docs, GitHub Advisory Database	https://docs.github.com/en/code-security/concepts/vulnerability-reporting-and-management/github-advisory-database	2026-11-03	Confirmed
150	actions/dependency-review-action README	https://github.com/actions/dependency-review-action	2026-11-03	Confirmed
151	GitHub CLI manual, gh auth logout	https://cli.github.com/manual/gh_auth_logout	2026-11-03	Confirmed
152	GitHub Docs, token expiration and revocation	https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/token-expiration-and-revocation	2026-11-03	Confirmed
153	GitHub REST API, revoke credentials	https://docs.github.com/en/rest/credentials/revoke	2026-11-03	Confirmed
154	GitHub Docs, OpenID Connect	https://docs.github.com/en/actions/concepts/security/openid-connect	2026-11-03	Confirmed
155	GitHub CLI manual, gh search repos	https://cli.github.com/manual/gh_search_repos	2026-11-03	Confirmed
156	PyPI docs, trusted publishers (with attestations)	https://docs.pypi.org/trusted-publishers/	2026-11-03	Confirmed
157	bundler-audit README	https://github.com/rubysec/bundler-audit	2026-11-03	Confirmed
158	cargo-audit README	https://github.com/rustsec/rustsec/tree/main/cargo-audit	2026-11-03	Confirmed
159	Microsoft Learn, auditing package dependencies (NuGet)	https://learn.microsoft.com/en-us/nuget/concepts/auditing-packages	2026-11-03	Confirmed
160	MITRE ATT&CK, S9008 Shai-Hulud	https://attack.mitre.org/software/S9008/	2026-11-03	Confirmed

#	Source	URL	Opened	Grade
161	MITRE ATT&CK, S9043 Mini Shai-Hulud	https://attack.mitre.org/software/S9043/	2026-11-03	Confirmed
162	MITRE ATT&CK, T1195.001	https://attack.mitre.org/techniques/T1195/001/	2026-11-03	Confirmed
163	MITRE ATT&CK, T1204.005	https://attack.mitre.org/techniques/T1204/005/	2026-11-03	Confirmed
164	MITRE ATT&CK, T1546.016	https://attack.mitre.org/techniques/T1546/016/	2026-11-03	Confirmed
165	MITRE ATT&CK, T1567.001	https://attack.mitre.org/techniques/T1567/001/	2026-11-03	Confirmed
166	TeamStarWolf library ATT&CK v20.0 data (`data/attack/*.jsonl`) and `mitre/` tree	https://github.com/TeamStarWolf/TeamStarWolf/tree/main/data/attack	2026-11-03	Confirmed
167	MITRE CAPEC-538	https://capec.mitre.org/data/definitions/538.html	2026-11-03	Confirmed
168	MITRE CAPEC-695	https://capec.mitre.org/data/definitions/695.html	2026-11-03	Confirmed
169	MITRE CAPEC-630	https://capec.mitre.org/data/definitions/630.html	2026-11-03	Confirmed
170	MITRE D3FEND API (Software Update and the other techniques named)	https://d3fend.mitre.org/api/technique/d3f:SoftwareUpdate.json	2026-11-03	Confirmed
171	MITRE D3FEND API, Trusted Library	https://d3fend.mitre.org/api/technique/d3f:TrustedLibrary.json	2026-11-03	Confirmed
172	OWASP Top 10 CI/CD Security Risks	https://owasp.org/www-project-top-10-ci-cd-security-risks/	2026-11-03	Confirmed
173	OWASP CICD-SEC-3, Dependency Chain Abuse	https://owasp.github.io/www-project-top-10-ci-cd-security-risks/CICD-SEC-03-Dependency-Chain-Abuse	2026-11-03	Confirmed
174	OWASP CICD-SEC-6, Insufficient Credential Hygiene	https://owasp.github.io/www-project-top-10-ci-cd-security-risks/CICD-SEC-06-Insufficient-Credential-Hygiene	2026-11-03	Confirmed
175	TeamStarWolf lab run: command output from a throwaway `node:22.11.0-bookworm-slim` container on a lab host (npm 10.9.0, Node v22.11.0), rendered as the three screenshots in Part 7	none (lab capture)	2026-11-04 (UTC)	Confirmed (lab)

Related references

- Threat Actors
- Malware Families
- Notable Incidents
- CVE Reference

- CWE Reference
- Supply Chain Security Reference
- DevSecOps Reference
- DevSecOps discipline
- Security Pipeline
- Secure Coding Reference
- JavaScript
- The XZ Utils Backdoor: A 2024 Supply-Chain Case Study
- AI Threats in the Cybersecurity Industry: A 2026 Case Study
- All case studies
- MITRE pages: S9008 Shai-Hulud, S9043 Mini Shai-Hulud, S9042 CanisterWorm, S9041 TeamPCP Cloud Stealer, S1246 BeaverTail, S1245 InvisibleFerret, G1056 TeamPCP, G0082 APT38, G1052 Contagious Interview, T1195.001, T1204.005, T1546.016, T1677, T1567.001, CAPEC-538, CAPEC-695